



ENSEIGNEMENT SUPERIEUR ET UNIVERSITAIRE
UNIVERSITE OFFICIELLE DE MBUJIMAYI(UOM)
Av. Kalonji, n°27, Q /Kansele, C/Muya, Mbuji-Mayi, Kasai oriental

Courriel : uomrecteur@gmail.com
Téléphone /WhatsApp : +243 85388663
www.uom.org

République Démocratique du Congo

FACULTE DES SCIENCES ET POLYTECHNIQUE

**LA REALISATION D'UNE PLATEFORME WEB MOBILE INTERACTIVE POUR LA
CONSULTATION DES RESULTATS DE LA DELIBERATION, INTEGRANT LA
MESSAGERIE ET FIL D'ACTUALITE.**

Présenté par : YOMBU MUDIANJI Mardochée

Promotion : L 3 informatique Lmd

*Directeur : Célestin TSHIMANGA
Professeur*

Co-directeur: Christy NTAMBWE

Master

Année Académique 2024 -2025

$$\sim \sim$$

EPIGRAPHE

L'ère de l'information transforme les
formes de pouvoir et influence.

Manuel Castells

~ || ~

DEDICACE

A mes très chers parents Jean de Dieu MUDIANJI et LILIANE LUSAMBA pour leur amour, leur attention, leur dévouement et tant de sacrifices consentis, en investissant sur ma modeste personne en termes d'une bonne éducation.

Mardoché YOMBU

REMERCIEMENT

Je tiens à remercier le Dieu tout puissant pour sa grâce qu'il a manifesté durant toutes ces années passées que son nom soit loué, Je remercie en second lieu mon directeur le professeur CELESTIN CIMANGA qui a voulu nous encadrer durant ce travail, je tiens également à remercier le Master Christy NTAMBUE qui nous a accompagné avec de remarques et correction jusqu'à la fin de ce travail.

Je veux être reconnaissant à ces gens qui sans leur contribution cet œuvre n'aurait pas dû existé, Nos remerciements s'adressent au comité de gestion de l'université officiel de Mbuji-Mayi, à mes professeurs, chefs de travaux, assistants, pour leur temps et leur expertise.

Nos remerciements s'adressent également aux gens qui sans eux j'aurais déjà arrêté à mi-chemin, ainsi nous citons ma famille, Jonathan tshiamala, Mardoché Mbenga, Josué mukuna, Rebecca KANYEBA et Naomie Musuamba, Exaucé Bankuna.

Enfin, mes amis et collègues Rufin Kazadi, Mardoche Ntambua, Josué kabanga, Jean mukendi, Mardoché kayombu, Pascal biaya, Keren Mianda et Sharon kabamba pour le soutien à notre égard au cours de notre formation, nous disons grandement merci pour votre concours inestimable. A tous ceux qui, sans doute diront que nous avons oublié de les citer, nous gardons dans notre cœur la chaleur que nous n'avons pas pu ressortir en les citant dans ces quelques lignes.

TABLE DE MATIERE

EPIGRAPHE.....	I
DEDICACE.....	I
REMERCIEMENT	III
TABLE DE MATIERE	IV
ABBREVIATIONS.....	VI
LISTE DES FIGURES.....	VII
LISTE DES TABLEAUX.....	VIII
INTRODUCTION GENERALE.....	1
PROBLEMATIQUE ET HYPOTHESES :.....	1
CHOIX ET INTERET DU SUJET	2
DELIMITATION DU TRAVAIL	2
METHODES ET TECHNIQUES UTILISEES	2
A. Méthode.....	2
B. Techniques.....	3
SUBDIVISIONS DU TRAVAIL	3
CHAPITRE I : GÉNÉRALITÉS SUR LE SYSTÈME D'INFORMATIONI .1.....	5
I.1 INTRODUCTION PARTIELLE.....	5
I.2 NOTION DE SYSTEME :	5
I.3 NOTION DE L'INFORMATION.....	7
I.4 SYSTÈME D'INFORMATION	9
I.5. DEVELOPPEMENT DU SYSTEME D'INFORMATION AUTOMATISE	13
I.6. METHODE DE CONCEPTION	13
I.7 CONCLUSION PARTIELLE.....	15
CHAPITRE II : L'ARCHITECTURE CLIENT SERVEUR.....	16
II .1 INTRODUCTION PARTIELLE	16
II.2 DEFINITION	16
II.3 HISTORIQUE ET EVOLUTION DE L'ARCHITECTURE CLIENT-SERVEUR	16
II.4 IMPORTANCE DE L'ARCHITECTURE CLIENT /SERVEUR.....	18
II.5 TYPES D'ARCHITECTURES CLIENT-SERVEUR	19
II.6 COMPARAISON DES DIFFERENTES ARCHITECTURES	22
II.7 COMPOSANTS D'UNE ARCHITECTURE CLIENT-SERVEUR :	22
II.8 FONCTIONNEMENT DE L'ARCHITECTURE CLIENT- SERVEUR.....	26
II.9 AVANTAGES.....	27
II.10 INCONVENIENTS.....	27
II.12 CONCLUSION PARTIELLE	28
CHAPITRE III : SERVICE WEB.....	29
III.1 INTRODUCTION PARTIELLE.....	29
III.2 DEFINITION.....	29
III.3 CONTEXTE HISTORIQUE	30
III.4 ROLE ET IMPORTANCE DES WEB SERVICES.....	30
III.5 CARACTERISTIQUES FONDAMENTALES DES WEB SERVICES.....	31

III.6 TYPES DE WEB SERVICES ET LEUR FONCTIONNEMENT	32
III.7 ARCHITECTURE DE COMMUNICATION ET PRINCIPES GENERAUX.....	39
III.8 CONCLUSION PARTIELLE.....	40
CHAPITRE IV : ETUDE PREALABLE ET MODELISATION.....	42
IV.1 INTRODUCTION PARTIELLE.....	42
IV.2 ETUDE PREALABLE.....	42
IV.3 DIAGNOSTIC DU PREABLE.....	44
IV.4 MODELISATION DU NOUVEAU SYSTEME.....	46
IV.5 CONCLUSION PARTIELLE	51
CHAPITRE V : DEPLOIEMENT	52
V.1 INTRODUCTION PARTIELLE	52
V.2 IMPLEMENTATION	52
V.2.1 Outils utilisés.....	52
V.2.2 Justification des choix techniques	53
V.2.3 Implémentation de la Base de Données :	53
V.2.4 Conception de l'Architecture de l'Application :	55
CONCLUSION PARTIELLE.....	61
CONCLUSION GENERALE	62
REFERENCES BIBLIOGRAPHIQUES.....	63

ABBREVIATIONS

http: Hyper Text Transfer Protocol

IA : Intelligence Artificielle

IOT: Internet of Things

IP : Internet Protocol

LAN : Réseau local

MCD : Modèle conceptuel de donnée

MERISE : Méthode d'étude et de réalisation Informatique pour les systèmes
D'entreprise

MLD : Modèle logique de donnée

MPD : Modèle physique de donnée

SGBD : Système de gestion de base de donnée

SI : Système d'information

SO : système opérant

SP : Système de pilotage

SQL: Structured Query Langage

TCP: Transmission Control Protocol

UI : Interface Utilisateur

UML : langage de modélisation unifié

LISTE DES FIGURES

Figure 1 . Structure du système.....	6
Figure 2. Architecture à 2 Niveaux	20
Figure 3 Architecture à 3 couches	20
Figure 4 Architectures N-couches	21
Figure 5 Schéma de circulation	47
Figure 6 Modele conceptuel de donnée	49
Figure 7 Modele logique de donnée	49
Figure 8 Création de la base de donnée sur phpAdmin	54
Figure 9 Création de la table Etudiant	54
Figure 10 .Création de la relation	55
Figure 11 Interface de démarrage et authentification	56
Figure 12 Interface de création du compte.....	57
Figure 13 Interface d'accueil et fil d'actualité	58
Figure 14 Interface de contact et de messagerie.....	59
Figure 15 Interface d'affichage de résultats	60
Figure 16 Interface administrateur	60
Figure 17 Interface de saisie de résultats	61

LISTE DES TABLEAUX

<i>Tableau 1 Avantages et Inconvénients du SOAP.....</i>	<i>34</i>
<i>Tableau 2 Avantages et Inconvénients du RestFul.....</i>	<i>38</i>
<i>Tableau 3 modèle physique de donnée.....</i>	<i>50</i>

INTRODUCTION GENERALE

Dans un monde où l'information circule à une vitesse fulgurante, il est essentiel que les institutions, mettent en place des stratégies de communication efficaces. Comme le souligne le sociologue Manuel Castells : « l'ère de l'information transforme les formes de pouvoir et influence » [1]. Pour les universités, cela signifie que la capacité à communiquer clairement et à publier les résultats des délibérations peut renforcer la légitimité et la confiance des étudiants et du personnel.

Pour cela, les technologies numériques offrent une opportunité sans précédent pour améliorer la transparence des informations au sein des institutions universitaires. En effet, comme le note Clay Shirky dans : « l'Internet est un amplificateur de voix qui permet aux citoyens d'exiger plus d'ouverture et de responsabilité » [2]. Les étudiants et le personnel s'attendent désormais à une communication ouverte et accessible.

PROBLEMATIQUE ET HYPOTHESES :

Dans ce contexte nous aborderons comment une plateforme web mobile interactive peut-elle répondre aux besoins des étudiants en matière de communication des programmes de cours, d'échange d'idées et de consultation des résultats académiques, tout en garantissant la clarté ?

De plus, comment l'intégration d'un fil d'actualité pourrait-elle faciliter l'accès à ces informations cruciales et encourager un engagement actif au sein de la communauté universitaire ?

Ces questions soulèvent des enjeux cruciaux concernant l'information sur les programmes de cours, les modes d'échange entre étudiants ainsi que la transparence dans la délibération des étudiants.

Dans la réalisation de ce travail nous envisageons de mettre en œuvre une plateforme web qui sera capable de faciliter l'entrée en communication avec l'ensemble

de la communauté estudiantine (Etudiants et personnels académiques, scientifiques et administratifs, techniques et ouvriers) ce qui permettra à plusieurs membres d'exprimer leurs idées en toute clarté et faciliter la consultation en temps réel des informations.

CHOIX ET INTERET DU SUJET

Le choix du sujet se porte sur La réalisation d'une plateforme web mobile interactive pour la consultation des résultats de la délibération, intégrant la messagerie et fil d'actualité.

Ce travail vise à doter à l'Université un canal de communication à large diffusion (Messagerie, fil d'actualité), et tout en facilitant aux étudiant l'accès aux résultats de la délibération d'une manière rapide et fiable.

DELIMITATION DU TRAVAIL

Notre travail ne se limitera qu'au niveau de la mise en place d'une application qui n'aura que 3 fonctionnalités à savoir : la consultation de résultats de la délibération, la messagerie et le fil d'actualité à l'université officielle de Mbuji-Mayi UOM, et les données utilisées couvrent une période allant de janvier à Août 2025.

METHODES ET TECHNIQUES UTILISEES

Comme le déclare Creswell : « Toutes les recherches, qu'elles soient qualitatives ou quantitatives, reposent sur des méthodes et techniques rigoureuses qui guident le chercheur dans la collecte et l'analyse des données » [3].

A. Méthode

Ainsi pour notre travail nous nous sommes servis de quatre méthodes à savoir :

➤ *Méthode structuro-fonctionnelle* : Elle nous a permis de connaître et comprendre la structure et le fonctionnement afin d'expliquer l'avancement et l'accomplissement de certaines tâches.

➤ *Méthode historique* : Celle qui nous a permis à récolter les informations concernant l'historique et la situation géographique de l'entreprise.

➤ *Méthode analytique* : Qui nous a permis de procéder par l'analyse des documents et étudier le fonctionnement de différents services de l'entreprise.

➤ *Méthode MERISE* : Celle-ci nous a aidé à formaliser notre problème, c'est-à-dire concevoir les différents modèles de données qui devront être implémentés dans la base de données de notre application.

B. Techniques

S'agissant des techniques, les techniques suivantes ont été utilisées :

➤ *La technique documentaire* : nous a aidé à consulter les documents de l'université officielle de Mbuji-Mayi comme par exemple le bulletin et relevé de côtes.

➤ *Technique d'interview* : elle nous a permise à questionner les différents membres en vue d'obtenir les informations pour l'élaboration de ce travail.

SUBDIVISIONS DU TRAVAIL

En dehors de la précédente Introduction et la conclusion qui suivra notre travail sera divisé en quatre chapitres suivants :

Chapitre 1 Généralités sur le système d'information : Dans ce chapitre nous allons définir quelques concepts de base du système d'information et aussi montrer son importance ainsi que les qualités d'un bon système d'information.

Chapitre 2 Architecture Clients serveurs : Dans ce chapitre nous allons décrire les concepts de base présenter, l'historique de l'architecture client serveurs, les types, les composants ainsi que les avantages et inconvénients.

Chapitre 3 Web service/service web : Ce chapitre parle sur le web service et décrit son historique, son rôle et importance jusqu'à la description des inconvénients ainsi que les avantages.

Chapitre 4 Etude Préalable et Modélisation : Dans ce chapitre, Nous allons démontrer l'étude faite du système existant et puis par les défaillances trouvées proposer une nouvelle modélisation du système.

Chapitre 5 Déploiement : Ici nous allons démontrer toutes les opérations et phase (méthodologique et technique) utilisé pour atteindre notre objectif.

CHAPITRE I : GÉNÉRALITÉS SUR LE SYSTÈME D'INFORMATIONI .1

I.1 INTRODUCTION PARTIELLE

Ce chapitre se concentrera sur l'étude des systèmes d'information au sein de l'entreprise, dans le but d'éclairer nos lecteurs sur les moments clés où l'informatique intervient concrètement dans la réalisation des diverses tâches organisationnelles.

Avant d'aborder l'étude du système d'information nous devons premièrement comprendre la signification de chaque mot qui le compose :

I.2 NOTION DE SYSTEME :

a. Définition : Le terme système est utilisé dans plusieurs disciplines (social, d'éducation, industriel, engineering, etc.). Plusieurs définitions existent pour définir un système. Cependant, tous les systèmes possèdent des propriétés communes. [4]

Un système est une collection organisée de personnes, de machines, de procédures, de documents, de données et d'autres entités interagissant entre elles et avec l'environnement pour atteindre des buts prédéfinis.

b. L'Entreprise vue en tant que système

Une entreprise est un groupement humain hiérarchisé qui met en œuvre des moyens intellectuels, physiques et financiers pour extraire, produire, transformer, distribuer des richesses conformément à des buts définis. [4]

Les circuits d'information et les cycles d'information au niveau d'une organisation permet de considérer l'entreprise comme un système complexe formé de trois sous-systèmes :

- ❖ Le système de décision (ou de pilotage)
- ❖ Le système d'information.
- ❖ Le système opérant.

Chaque système apporte des services à l'autre, le figure ci bat illustre cela

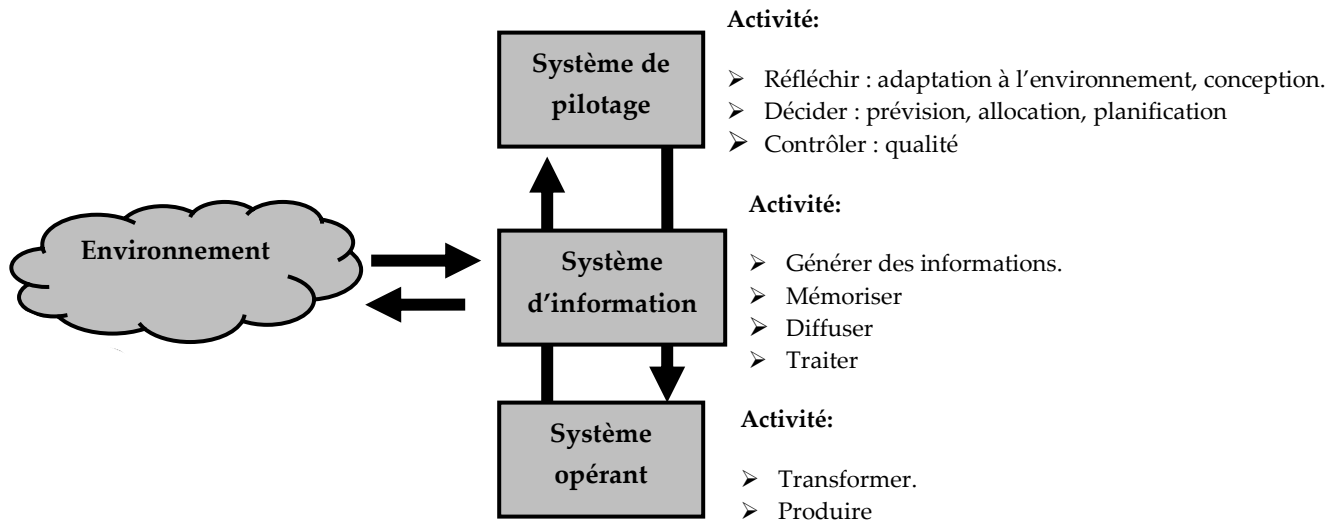


Figure 1 . Structure du système

Le système de pilotage (ou de décision) (SP) : Il se préoccupe de :

- Exploite les informations qui circulent
- Organise le fonctionnement du système
- Décide des actions à conduire sur le système opérant
- Raisonne en fonction des objectifs et des politiques de l'entreprise.

Le système opérant (SO)

- Reçoit les informations émises par le système de pilotage
- Se charge de réaliser les tâches qui lui sont confiées
- Génère à son tour des informations en direction du système de pilotage (qui peut ainsi contrôler les écarts et agir en conséquence).
- Il englobe toutes les fonctions liées à l'activité propre de l'entreprise (facturer les clients, régler les salaires, gérer les stocks, ...).

Le système d'information (SI) : Les grandes fonctions du SI d'une organisation sont :

- Générer les informations représentant l'activité du système opérant au sein de son environnement (les informations primaires) Recevoir des

informations de l'intérieur ou de l'extérieur de l'organisation de les traiter puis de les retransmettre vers l'intérieur ou l'extérieur.

- Mémoriser des informations (Pour comparer, prévoir, ...).
- Diffuser l'information.
- Assurer l'accès à la mémoire et la communication des informations.
- Réaliser tous les traitements nécessaires au fonctionnement du système.

I.3 NOTION DE L'INFORMATION

En informatique, l'information est un élément de connaissance (voix, donnée, image) susceptible d'être conservé, traité ou transmis.

Elle représente une interprétation significative d'un ensemble de données, rendant ces données utiles et pertinentes pour un objectif donné. C'est la brique de base de tout système informatique, le carburant qui alimente les algorithmes et les processus.

a. Différence entre donnée et information [6]

❖ Données : Ce sont des faits bruts, des symboles, des chiffres, des lettres ou des signaux qui n'ont pas encore été organisés ou interprétés pour obtenir un sens.

Elles sont souvent considérées comme des éléments atomiques. Par exemple : `165`, `rouge`, `pomme`, `3.14`.

❖ Information : C'est le résultat du traitement, de l'organisation, de la structuration et de l'interprétation des données.

L'information confère un sens et un contexte aux données, les rendant utiles pour la prise de décision ou la compréhension.

En reprenant l'exemple ci-dessus, si on organise ces données et qu'on leur donne un contexte, on pourrait obtenir l'information suivante : "Une pomme rouge pèse 165 grammes et son prix est de 3,14 dollars."

b. Les sources d'information :

❖ Interne : rapports des dirigeants, documents comptables et financiers, notes de services, documents sociaux (documents issus des assemblées générales),

études commerciales, éléments juridiques, informations sociales des salariés (journaux d'entreprises), etc...

❖ Externe : médias, documents spécialisés (fiches techniques), organisations publiques (administrations, ministères), institutions spécialisées (INSEE), institutions internationales (ONU, OCDE)

c. Les formes de l'information :

- Ecrite : une lettre, une facture, une fiche de stock, une note interne...
- Picturale : Schéma d'un circuit imprimé, graphique des ventes, photo d'un ordinateur, films publicitaires...
- Orale ou sonore : Discussion entre collègues, conversation téléphonique, réunion, "bips" d'un ordinateur...
- Autres formes : tactiles, olfactives...

d. Qualités de l'information

Pour être fonctionnelle, l'information doit être :

- Fiable : Issue de sources sûres, vérifiées. Une information erronée peut conduire à de graves problèmes.
- Pertinente : Info adaptée au décideur
- Utile : En rapport avec le problème traité
- Exacte : les données ne comportent pas d'erreur
- Précise : par rapport au problème à traiter, l'info est suffisante pour réduire les risques d'erreur.
- Disponible : le décideur peut utiliser l'info au moment où il en a besoin
- Actuelle : l'info appartient à la réalité la plus récente

e. Rôles de l'information :

○ Carburant des Traitements et des Algorithmes : L'information, sous forme de données structurées ou non, est la matière première que les ordinateurs et les logiciels

traitent. Sans information à manipuler, les algorithmes n'auraient aucune utilité. Elle alimente les calculs, les analyses, les simulations et toutes les opérations logiques.

- Base de la Prise de Décision : L'informatique a pour objectif de transformer des données brutes en informations significatives. Ces informations, une fois traitées et organisées, deviennent la base sur laquelle les utilisateurs (individus, entreprises, gouvernements) prennent des décisions éclairées. Les tableaux de bord, les rapports et les analyses prédictives sont des exemples de la façon dont l'information est présentée pour faciliter la décision.

- Moyen de Communication : L'information est le contenu échangé via les réseaux informatiques. Que ce soit des e-mails, des messages instantanés, des pages web, des vidéos en streaming ou des transactions financières, l'informatique permet la transmission rapide et efficace de l'information à travers le monde.

- Fondement de la Connaissance et de l'Intelligence Artificielle : Au-delà de la simple information, l'informatique vise à extraire de la connaissance et de l'intelligence. Les systèmes d'IA et d'apprentissage automatique apprennent à partir de grandes quantités d'informations (données d'entraînement) pour identifier des modèles, faire des prédictions et même générer de nouvelles informations. En somme, l'information est le cœur battant de l'informatique. Elle est à la fois la matière première, le produit fini et le vecteur qui permet aux systèmes informatiques de fonctionner, de communiquer, d'analyser et de soutenir l'intelligence humaine et artificielle.

I.4 SYSTÈME D'INFORMATION

- a. **Système d'entreprise** : un système d'entreprise est un ensemble organisé d'éléments (ressources humaines, financières, matérielles) qui interagissent de manière à produire des résultats spécifiques. [4]

L'analyse systémique de l'entreprise correspond à une approche en termes de flux visant à mettre en évidence les interactions à l'intérieur de l'entreprise entre le système d'information, le système de décision et le système opérant et les échanges entre l'entreprise et son environnement extérieur. On distingue deux types de flux :

- Les flux physiques qui correspondent soit à des flux logistiques (matière 1re, en-cours de production, produits finis, etc.), soit à des flux financiers.
- Les flux d'information qui traduisent les divers échanges d'information entre l'entreprise et son environnement extérieur ou des transferts d'information au sein de l'organisation.

b. Système d'information

Le système d'information (SI) peut être défini comme un ensemble organisé de ressources (matériel, logiciel, personnel, données, procédures...) permettant d'acquérir, de stocker, de traiter, de communiquer des informations de toutes formes dans une organisation. [7]

I.4.1. Fonctions du Système d'Information :

Les principales fonctions d'un système d'information (SI) sont :

1. Collecte de Données : La collecte de données est la première étape du processus d'un système d'information. Cela implique la collecte d'informations pertinentes à partir de diverses sources, qui peuvent être internes (comme les transactions commerciales) ou externes (comme des études de marché).

Elle garantit que l'organisation dispose des données nécessaires pour prendre des décisions éclairées et peut aussi impliquer des méthodes variées telles que des enquêtes, des observations, ou l'extraction de données à partir d'autres systèmes.

2. Stockage et Récupération des Informations : Une fois les données collectées, elles doivent être stockées de manière sécurisée et organisée. Cela se fait généralement dans des bases de données ou des systèmes de gestion de contenu.

Il est nécessaire de mieux stocker mes données car ceci permet un accès rapide et sans difficulté aux informations.

3. Traitement et Analyse des Données : Le traitement des données implique la transformation des données brutes en informations significatives. Cela peut inclure le tri, le filtrage, l'agrégation et l'analyse statistique.

Cette fonction permet aux organisations de tirer des conclusions exploitables à partir des données tout en révélant des tendances, identifier des problèmes potentiels et fournir un aperçu précieux pour la prise de décision stratégique.

4. Diffusion d'Informations : La diffusion consiste à communiquer les informations traitées aux utilisateurs concernés au sein de l'organisation. Cela peut se faire par divers moyens, tels que les rapports, les tableaux de bord, ou même par e-mail.

I.4.2. Rôle du système d'information

Le système d'information joue plusieurs rôles à savoir :

a. Support aux Opérations : Les systèmes d'information facilitent les opérations quotidiennes en automatisant des tâches répétitives et en améliorant l'efficacité.

Par exemple : Suivi des Transactions : Les systèmes de point de vente (POS) permettent aux entreprises de gérer les ventes, les inventaires et les services clients en temps réel.

b. Prise de Décision : Un SI permet aux décideurs d'accéder à des informations pertinentes et précises pour prendre des décisions éclairées.

c. Communication et Collaboration : Les systèmes d'information améliorent la communication interne et externe grâce à l'Intranet et l'extranet.

I.4.3. Qualité du système d'information

Les qualités du système d'information déterminent directement sa capacité à soutenir les objectifs stratégiques et opérationnels nous pouvons citer :

1. Pertinence et Adéquation : Un bon SI doit être pertinent par rapport aux besoins de l'organisation. Cela signifie qu'il doit collecter, traiter et diffuser les informations qui

sont réellement utiles aux décideurs et aux utilisateurs. Il doit aussi être ****adéquat**** aux processus métier de l'entreprise, les soutenant plutôt que de les entraver.

2. *Fiabilité et Précision* : La fiabilité est cruciale. Les données et les informations générées par le SI doivent être exactes et sans erreur. Des informations erronées peuvent mener à de mauvaises décisions, des pertes financières ou une dégradation de la réputation. La précision des données à l'entrée est le fondement de la fiabilité à la sortie.

3. *Disponibilité et Accessibilité* : Le SI doit être disponible quand les utilisateurs en ont besoin. Cela implique une infrastructure robuste, des sauvegardes régulières et des plans de reprise après sinistre. L'accessibilité garantit que les utilisateurs autorisés peuvent facilement accéder aux informations pertinentes, quel que soit leur emplacement ou l'appareil qu'ils utilisent.

4. *Sécurité* : La sécurité est une qualité non négociable. Un SI doit protéger les informations sensibles contre les accès non autorisés, les modifications, les destructions ou les divulgations. Cela inclut la sécurité physique, la sécurité logique (pare-feu, antivirus, gestion des accès) et la protection contre les cyberattaques.

5. *Flexibilité et Évolutivité* : Un SI doit être flexible pour s'adapter aux changements futurs de l'organisation, des marchés ou des technologies. Il doit également être évolutif, capable de gérer une croissance du volume de données, du nombre d'utilisateurs ou de la complexité des processus sans dégrader ses performances.

6. *Efficacité et Performance* : L'efficacité d'un SI se mesure à sa capacité à optimiser les processus, réduire les coûts et améliorer la productivité. La performance fait référence à la rapidité avec laquelle le système exécute les tâches, traite les requêtes et fournit les informations. Un système lent peut freiner les opérations.

8. *Intégration* : La capacité d'un SI à s'intégrer avec d'autres systèmes, qu'ils soient internes ou externes, est primordiale. L'intégration permet le partage fluide des données, évitant la duplication et assurant une vue d'ensemble cohérente des informations.

9. *Conformité* : Un SI doit être conforme aux réglementations légales, aux normes de l'industrie et aux politiques internes de l'entreprise (par exemple, RGPD pour la protection des données).

10. *Coût-efficacité* : Enfin, un SI doit offrir un bon rapport coût-efficacité. Le coût total de possession (développement, maintenance, mises à niveau, formation) doit être justifié par les bénéfices qu'il apporte à l'organisation. [8]

I.5. DEVELOPPEMENT DU SYSTEME D'INFORMATION AUTOMATISE

Développer un système d'information par exemple de gestion des ressources humaines, c'est mener un ensemble d'activités complexes et variées des décisions et des planifications, d'analyse de la réalité et de recueil des besoins informationnels, de conception et d'évaluation du système, de réalisation des tests, de mise en orientation et de contrôle de qualité qui aboutissent à faire passer le système d'information d'une situation initiale de profit à une situation finale de systèmes concrets et opérationnels.

Il est désormais professionnellement admis que le processus de développement de tout système d'information s'organise globalement en deux phases : la première phase qui aboutit à la **spécification** du système d'information sous la forme d'un schéma et une grande phase de **réalisation** qui conduit à la fabrication effective du système d'information.

I.6. METHODE DE CONCEPTION

La *conception de systèmes d'information est une tâche* complexe et de haut niveau qui nécessite un bon pouvoir d'abstraction et la prise en compte d'un grand nombre de contraintes (*de coût, d'existant, ...*) et d'un grand nombre de personnes (*direction, service informatique, services, utilisateurs*)

Il sied de noter que plusieurs méthodes de conception existent ; certaines méthodes disparaissent et laissent la place à d'autres méthodes plus adaptées. D'autres méthodes évoluent dans le temps en fonction des différentes fonctionnalités.

Chaque méthode a ses qualités et ses défauts. Chaque méthode est adaptée au type de projet (Objet, industriel, gestion) et aux outils (SGBD, LANGAGE DE PROGRAMMATION)

Quelques méthodes connues sont :

⇒ MERISE 1 et MERISE 2 qui sont des méthodes systémiques

⇒ SADT : La méthode SADT (**Structured Analysis and Design Technique**) est un outil graphique associé à une méthode d'analyse descendante modulaire et hiérarchisée (Design se traduit ici par conception).

⇒ SS ADM (**Structured Systems Analysis and Design Method**) : La **méthode SSADM** (Analyse structurée des systèmes et de la méthode de conception) est une autre méthode traitant de la conception des systèmes d'information. Il a été développé au Royaume-Uni et a été traditionnellement utilisé pour le développement des systèmes moyennes ou grandes entreprises.

NB : UML qui est aussi beaucoup utilisé aujourd'hui n'est pas tout à fait une méthode, mais un langage de modélisation Unifié.

Toute méthode doit mettre en œuvre quatre composants indissociables et complémentaires :

⇒ **Des modèles** : chaque méthode est basée sur au moins un modèle. Celui-ci est vu ici comme un ensemble des concepts et des règles pour les utiliser, destinés soit à expliquer et construire la représentation du phénomène organisationnel, soit à expliquer et représenter les éléments qui composent le SI et leurs relations.

⇒ **Des langages** : ensemble des constructions qui permettent de décrire formellement les spécifications du SI élaborées aux différents stades du processus de conception, s'appuyant éventuellement sur un ou des modèles de la méthode.

⇒ **Une démarche** : le processus opératoire grâce auquel s'effectue le travail de modélisation, de description, d'évaluation et de réalisation du système d'information.

⇒ *Des outils* : les logiciels qui supportent la démarche.

I.7 CONCLUSION PARTIELLE

Le Système d'Information (SI) constitue l'épine dorsale de toute organisation moderne. Défini comme un ensemble structuré de ressources (matériel, logiciel, personnel, données, procédures), son rôle fondamental est d'acquérir, stocker, traiter et communiquer l'information sous toutes ses formes.

Le SI est au cœur de la performance organisationnelle. Pour parler d'un bon système d'information, il doit impérativement posséder des qualités intrinsèques telles que la pertinence, la fiabilité, la disponibilité, la sécurité, la flexibilité, l'efficacité, l'intégration et la conformité. En somme, un SI de qualité est un atout stratégique qui permet à l'entreprise non seulement de fonctionner, mais aussi de s'adapter, d'innover et de prospérer dans un environnement complexe et en constante évolution.

CHAPITRE II : L'ARCHITECTURE CLIENT SERVEUR

II.1 INTRODUCTION PARTIELLE

Avec cette progression constante de la technologies les utilisateurs ont besoin d'accéder au divers types de données à n'importe quel moment et tout cela dans un délai bien déterminé, C'est ainsi que l'architecture client-serveur est un modèle fondamental en informatique distribuée qui régit la manière dont les différents composants d'un système interagissent entre eux via un réseau, que ce soit pour naviguer sur le web, envoyer des e-mails, ou utiliser des applications en ligne.

II.2 DEFINITION

a) **L'architecture client-serveur** : L'architecture client-serveur est un modèle de conception informatique qui divise les tâches ou les charges de travail entre les fournisseurs de ressources ou de services, appelés serveurs, et les demandeurs de ces ressources, appelés clients. [9]

Dans ce modèle, le client envoie des requêtes au serveur, qui traite ces requêtes et renvoie des réponses appropriées. Cette architecture repose sur une séparation claire des rôles à savoir :

- **Client** : C'est l'application ou le dispositif (comme un ordinateur, un smartphone ou une tablette) qui demande des services ou des données. Le client est souvent l'interface utilisateur avec laquelle l'utilisateur interagit.

- **Serveur** : C'est la machine ou le système qui fournit les ressources, stocke les données et exécute des applications. Le serveur peut gérer plusieurs clients à la fois.

II.3 HISTORIQUE ET EVOLUTION DE L'ARCHITECTURE CLIENT-SERVEUR

II.3.1 Origines de l'Architecture Client-Serveur

L'architecture client-serveur a émergé dans les années 1980, en réponse à la nécessité croissante de gérer des applications plus complexes et des réseaux de plus en plus interconnectés. Avant cette époque, la plupart des systèmes informatiques étaient basés sur des architectures monolithiques, où toutes les fonctions d'application, y

compris le traitement des données et l'interface utilisateur, étaient exécutées sur une seule machine.

Les premiers systèmes client-serveur ont été développés pour améliorer cette approche en séparant les responsabilités. Par exemple :

- **Systèmes à distance** : Les premiers concepts de client-serveur ont été influencés par les systèmes à distance, où plusieurs utilisateurs pouvaient accéder à un même serveur central via des terminaux. Cela a permis une centralisation des ressources tout en offrant une interface utilisateur décentralisée.

- **Réseaux locaux (LAN)** : Avec l'avènement des réseaux locaux dans les années 1980, il est devenu possible pour plusieurs ordinateurs (clients) de se connecter à un serveur centralisé pour accéder à des ressources partagées comme des fichiers et des imprimantes.

II. 3.2 Évolution à Travers le Temps

L'architecture client-serveur a connu plusieurs phases d'évolution au fil du temps :

- *Client-Serveur Traditionnel (Années 1980-1990)* : À cette époque, les clients étaient souvent des ordinateurs personnels exécutant des applications graphiques, tandis que les serveurs étaient généralement des machines puissantes hébergeant des bases de données et traitant les requêtes.

Les protocoles comme SQL pour les bases de données et TCP/IP pour la communication réseau ont été standardisés.

- *Architecture N-Tiers (Années 1990)* : L'architecture s'est étendue au modèle N-tiers, où la logique d'application a été séparée en plusieurs couches (présentation, logique métier, accès aux données). Cela a permis une plus grande modularité et facilité de maintenance.

Les serveurs d'applications sont devenus courants, gérant la logique métier et déchargeant le serveur de base de données.

– *Web et Services Internet (Années 2000)* : L'explosion d'Internet a conduit à la popularité des applications web basées sur le modèle client-serveur. Les navigateurs Web sont devenus les clients principaux, interagissant avec des serveurs web via HTTP.

L'essor du cloud computing a également transformé ce modèle, permettant aux entreprises de déployer leurs applications sur des infrastructures distantes.

– *Microservices et Architectures Distribuées (Années 2010)* : Récemment, l'architecture microservices a gagné en popularité, permettant aux développeurs de créer des applications modulaires où chaque service est indépendant mais peut communiquer avec d'autres services via API.

Cela favorise une plus grande agilité dans le développement et le déploiement d'applications.

– *Émergence du Cloud et de l'Edge Computing* : Aujourd'hui, nous voyons une intégration croissante du cloud computing et de l'edge computing dans les architectures client-serveur. Les données peuvent être traitées près de leur source (edge) ou stockées dans le cloud pour un accès facile.

Cette évolution répond aux besoins croissants en matière de latence faible, d'évolutivité et de traitement en temps réel.

- Exemples : Google Chrome, Mozilla Firefox.

II.4 IMPORTANCE DE L'ARCHITECTURE CLIENT /SERVEUR

L'architecture client-serveur est d'une grande importance dans le paysage technologique moderne pour plusieurs raisons :

1. Scalabilité : Elle permet aux systèmes de s'adapter facilement à une augmentation du nombre d'utilisateurs ou à une augmentation de la charge en ajoutant simplement plus de serveurs ou en optimisant ceux qui existent.

2. Centralisation des Données : Les données sont généralement stockées sur le serveur, ce qui facilite leur gestion, leur sauvegarde et leur sécurité. Cela réduit également la redondance des données sur différents clients.

3. Flexibilité et Maintenance : Les mises à jour peuvent être effectuées sur le serveur sans nécessiter de modifications sur chaque client, ce qui simplifie la maintenance et améliore l'expérience utilisateur.

4. Accès à Distance : Avec l'avènement d'Internet et du cloud computing, les utilisateurs peuvent accéder aux services depuis n'importe où dans le monde, rendant cette architecture particulièrement pertinente pour les applications web et mobiles.

5. Interopérabilité : Grâce à l'utilisation de protocoles standards, différents systèmes peuvent interagir facilement entre eux, permettant aux entreprises d'intégrer diverses technologies.

II.5 TYPES D'ARCHITECTURES CLIENT-SERVEUR

L'architecture client-serveur se décline en plusieurs modèles, classifiés généralement selon le nombre de "niveaux" (tiers) ou de couches logiques impliqués dans le traitement d'une requête.

Cette classification reflète la répartition des responsabilités entre les clients et les serveurs. Parmi les types d'architectures clients/serveur nous citons :

II.5.1 Architecture à Deux Niveaux (2-Tiers / Two-Tier Architecture)

L'architecture à 2 couches, également connue sous le nom d'architecture client-serveur classique, se compose de deux parties principales : le client et le serveur.

- Client : Il s'agit de l'interface utilisateur où les utilisateurs interagissent avec l'application.
- Serveur : Il gère la logique métier et l'accès aux données.

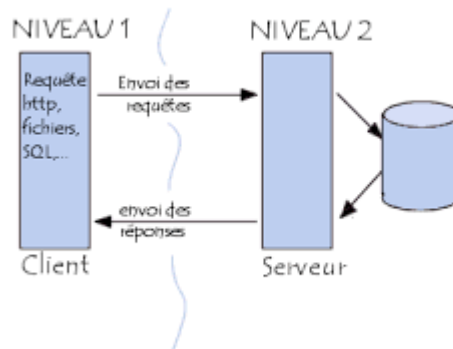


Figure 2. Architecture à 2 Niveaux

A. Avantages :

- Simplicité : Facile à mettre en œuvre et à comprendre.
- Performances : Moins de latence pour les petites applications.

B. Inconvénients :

- Scalabilité limitée : Difficulté à gérer une augmentation du nombre d'utilisateurs.
- Monolithisme : Tous les composants sont souvent étroitement couplés.

II.5.2 Architecture à 3 couches

L'architecture à 3 couches ajoute une couche intermédiaire entre le client et le serveur, souvent appelée "couche de logique métier".

- Couche de présentation (client) : Interface utilisateur.
- Couche de logique métier (serveur intermédiaire) : Traite la logique d'application.
- Couche de données (base de données) : Gère le stockage des données.

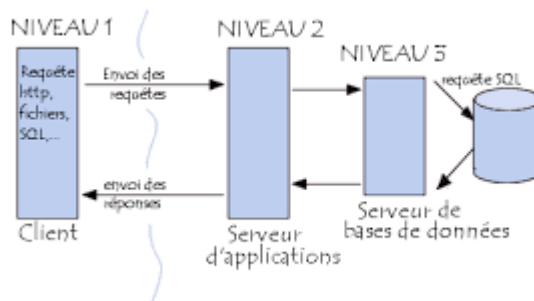


Figure 3 Architecture à 3 couches

A. Avantages :

- Meilleure séparation des préoccupations : Facilite la maintenance et les mises à jour.
- Scalabilité améliorée : Chaque couche peut être mise à l'échelle indépendamment.

B. Inconvénients :

- Complexité accrue : Nécessite plus de ressources et une gestion plus complexe.
- Latence potentiellement plus élevée en raison de la communication entre les couches.

II.5.3 Architectures N-couches et microservices

Les architectures N-couches étendent le concept des architectures à plusieurs couches, permettant une modularité encore plus grande. Les microservices sont un exemple moderne où chaque service est indépendant et peut être développé, déployé et mis à l'échelle séparément.

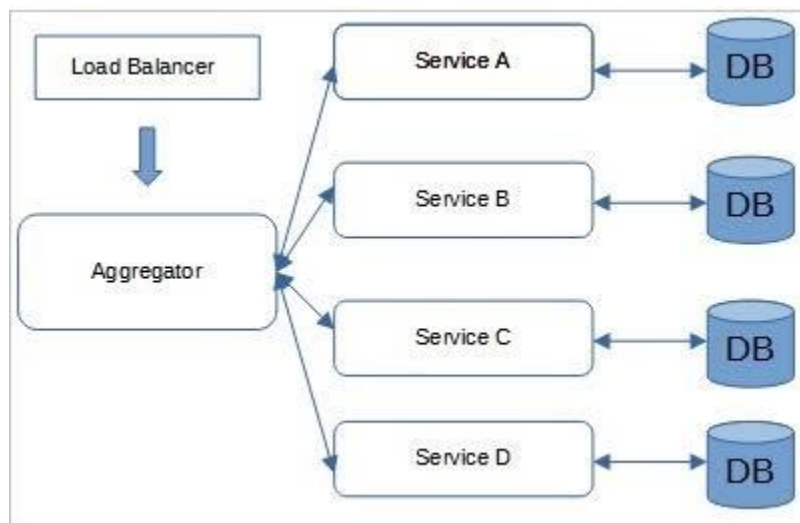


Figure 4 Architectures N-couches

A. Avantages :

- Flexibilité : Chaque micro service peut utiliser différentes technologies adaptées à ses besoins spécifiques.
- Résilience : En cas de défaillance d'un service, les autres peuvent continuer à fonctionner.

B. Inconvénients :

- Complexité de gestion : Nécessite une orchestration pour gérer les multiples services.
- Dépendances inter-services : La communication entre services peut introduire des latences.

II.6 COMPARAISON DES DIFFERENTES ARCHITECTURES

Lorsqu'on compare ces architectures, plusieurs facteurs doivent être pris en compte :

1. Simplicité vs Complexité : Les architectures à 2 couches sont simples, tandis que les microservices sont complexes mais offrent une flexibilité accrue.
2. Scalabilité: Les architectures à 3 couches et N-couches permettent une meilleure scalabilité que celles à 2 couches.
3. Maintenance: Les systèmes modulaires comme les microservices sont généralement plus faciles à maintenir que les systèmes monolithiques. [10]

II.7 COMPOSANTS D'UNE ARCHITECTURE CLIENT-SERVEUR :

Voici les principaux composants d'une architecture client-serveur :

II.7.1 Le Client (Front-end)

Le client est l'application ou le dispositif qui initie la demande de service et interagit directement avec l'utilisateur. C'est le point d'entrée de l'utilisateur dans le système.

Il est souvent caractérisé par :

- **Interface Utilisateur (UI)** : Présente les informations à l'utilisateur et lui permet d'interagir avec l'application (boutons, formulaires, affichage de données).
- **Logique de présentation** : Gère la manière dont les données reçues du serveur sont affichées.
- **Envoi des requêtes** : Formate et envoie les requêtes au serveur via le réseau.
- **Réception et traitement des réponses** : Reçoit les réponses du serveur et les interprète pour les afficher.

II.7.1.A Types de clients : Un client peut être :

- *Navigateurs Web* : Chrome, Firefox, Safari (pour les applications web).
- *Applications Mobiles* : Applications iOS ou Android sur smartphones et tablettes.
 - *Applications de Bureau (Desktop Apps)* : Logiciels installés sur un ordinateur (ex : Microsoft Word se connectant à OneDrive, un client e-mail comme Outlook).
 - *Autres dispositifs* : Terminaux de point de vente (POS), objets connectés (IoT), autres serveurs agissant comme clients (pour des services web).

II.7.2 Le Serveur (Back-end)

Le serveur est l'application ou le système qui fournit le service demandé par le client. C'est le "cerveau" de l'application, où résident la logique métier et les données.

Il est souvent caractérisé par :

- *Écoute des requêtes* : Le serveur est constamment en attente de requêtes provenant des clients sur un port réseau spécifique.
- *Traitement des requêtes* : Reçoit, analyse et exécute la logique nécessaire pour répondre à la demande du client. Cela peut impliquer des calculs, des validations, des accès à des bases de données, etc.
- *Gestion des ressources* : Alloue des ressources (CPU, mémoire, I/O) pour traiter les requêtes.

- *Envoi des réponses* : Formate et envoie la réponse au client.
- *Sécurité* : Gère l'authentification des clients, l'autorisation d'accès aux ressources et la protection des données.
- *Scalabilité* : Les serveurs sont souvent conçus pour gérer un grand nombre de requêtes simultanées et peuvent être mis à l'échelle (verticalement en augmentant la puissance, ou horizontalement en ajoutant plus de serveurs).

II.7.2.A Types de serveurs : Nous distinguons plusieurs types de serveur par leur fonction, à savoir :

- *Serveurs Web* : Servent des pages web et des ressources statiques (HTML, CSS, JavaScript, images). Ex: Apache HTTP Server, Nginx, Microsoft IIS.
- *Serveurs d'Applications* : Exécutent la logique métier complexe et interagissent avec les bases de données. Ex: Apache Tomcat, JBoss/WildFly, Node.js avec Express, Python avec Django/Flask.
- *Serveurs de Base de Données* : Stockent, organisent et gèrent les données. Ex: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, MongoDB.
- *Serveurs de Fichiers* : Stockent et partagent des fichiers.
- *Serveurs de Messagerie* : Gèrent l'envoi et la réception d'e-mails.

II.7.3 Le Réseau

Le réseau est l'infrastructure de communication qui permet aux clients et aux serveurs d'échanger des informations. Sans un réseau fonctionnel, l'architecture client-serveur ne peut exister. Il sert de :

- *Transport des données* : Permet la transmission des requêtes et des réponses entre les clients et les serveurs.
- *Protocoles* : Utilise des ensembles de règles (protocoles) pour garantir une communication structurée et fiable ; comme :

- TCP/IP (Transmission Control Protocol/Internet Protocol) : La suite de protocoles fondamentale d'Internet, assurant la livraison fiable des paquets de données et l'adressage.

- HTTP/HTTPS (Hypertext Transfer Protocol/Secure) : Le protocole le plus courant pour la communication sur le web.

- FTP (File Transfer Protocol) : Pour le transfert de fichiers.

- SMTP/POP3/IMAP : Pour l'e-mail.

Équipements réseau : Routeurs, commutateurs (switches), pare-feu, câbles, ou connexions sans fil (Wi-Fi, 4G/5G).

II.7.4 Middleware (surtout dans les architectures 3-tiers ou N-tiers)

Le middleware est un logiciel intermédiaire qui facilite la communication et la gestion des données entre différents composants d'un système distribué. Il peut s'agir de serveurs d'applications, de passerelles de messages, de courtiers de messages, etc.

Son rôle est :

- *Centralisation de la logique métier* : Dans une architecture 3-tiers, le middleware (serveur d'application) contient la logique métier, ce qui rend le client plus léger et plus facile à maintenir.
- *Gestion des transactions* : Assure la cohérence des opérations complexes sur plusieurs ressources.
- *Sécurité et authentification* : Peut gérer les politiques de sécurité avant de transmettre les requêtes aux serveurs de données.
- *Optimisation des performances* : Peut inclure des mécanismes de mise en cache, de répartition de charge (load balancing) pour améliorer la réactivité.
- *Connectivité* : Permet aux clients de différents types de se connecter à des services serveur variés sans avoir à connaître les détails de leur implémentation.

Exemples : Serveurs d'applications (Java EE, .NET), Message Queues (RabbitMQ, Apache Kafka), Enterprise Service Buses (ESB). [11]

II.8 FONCTIONNEMENT DE L'ARCHITECTURE CLIENT- SERVEUR

Le principe de base est la répartition des tâches entre deux types d'entités :

1. Le Client : C'est le programme ou le dispositif qui initie la communication et demande un service. Le client peut être un navigateur web, une application de messagerie, une application mobile, etc. Il est généralement en charge de l'interface utilisateur et de la présentation des données. Le client envoie des requêtes au serveur pour obtenir des informations ou pour effectuer des actions.

2. Le Serveur : C'est le programme ou le dispositif qui écoute les requêtes des clients, traite ces requêtes et renvoie une réponse. Le serveur est généralement une machine plus puissante, dédiée au stockage des données, à l'exécution de la logique métier de l'application et à la gestion des ressources. Un serveur peut gérer plusieurs clients simultanément.

Le processus de communication se déroule généralement comme suit :

1. Requête du client : Le client envoie une requête au serveur. Cette requête contient des informations sur le service demandé (par exemple, "afficher la page web X", "envoyer cet e-mail", "récupérer les données de l'utilisateur Y").

2. Traitement par le serveur : Le serveur reçoit la requête du client. Il la traite en accédant à ses propres ressources (base de données, fichiers, autres services), en exécutant des calculs ou en appliquant une logique métier.

3. Réponse du serveur : Une fois le traitement terminé, le serveur envoie une réponse au client. Cette réponse peut contenir les données demandées, une confirmation de l'action, un message d'erreur, etc.

4. Affichage par le client : Le client reçoit la réponse du serveur et l'affiche à l'utilisateur de manière appropriée (par exemple, affiche la page web, confirme l'envoi de l'e-mail).

II.9 AVANTAGES

- Centralisation des Données et de la Gestion : Les données et les ressources critiques sont stockées et gérées sur le serveur ;cela facilite la sauvegarde, la sécurité, l'intégrité et la cohérence des données. La mise à jour des applications ou des données est simplifiée car elle ne doit être effectuée qu'à un seul endroit (le serveur).

- Sécurité Améliorée : La gestion des accès, l'authentification et l'autorisation peuvent être centralisées et appliquées de manière cohérente sur le serveur. Les clients n'ont pas un accès direct aux données brutes ;cela réduit les risques d'accès non autorisé et de corruption des données.

- Scalabilité : Possibilité d'augmenter la capacité du système en ajoutant plus de serveurs (scalabilité horizontale) ou en augmentant la puissance des serveurs existants (scalabilité verticale).Permet de répondre à un nombre croissant d'utilisateurs et de requêtes sans sacrifier les performances.

- Partage des Ressources :Les clients peuvent partager des ressources coûteuses ou limitées (imprimantes, bases de données, applications logicielles) gérées par le serveur. Réduit les coûts matériels et logiciels par utilisateur.

- Hétérogénéité : Les clients et les serveurs peuvent être développés avec différentes technologies, systèmes d'exploitation et matériels, tant qu'ils respectent les protocoles de communication standardisés. Flexibilité dans le choix des technologies et des plateformes.

II.10 INCONVENIENTS

- Dépendance au Serveur (Single Point of Failure) :Si le serveur principal tombe en panne, tous les clients qui dépendent de ce serveur perdent l'accès aux services. Peut entraîner une interruption de service et une perte de productivité. Nécessite des mécanismes de haute disponibilité (redondance, clustering).

- Coût d'Infrastructure : La mise en place et la maintenance de serveurs puissants, de systèmes de stockage, de solutions de réseau et de sécurité peuvent être coûteuses. investissement initial et coûts opérationnels élevés.
- Complexité de Développement et de Déploiement : Concevoir et développer des applications distribuées avec plusieurs composants (client, serveur, base de données, middleware) est plus complexe que des applications monolithiques. Le déploiement et la gestion des versions peuvent également être un défi ; Nécessite des compétences spécialisées et une planification rigoureuse.
- Latence Réseau : La communication entre le client et le serveur via le réseau introduit une latence (délai) inhérente due au temps de transmission des données. Peut affecter la performance perçue par l'utilisateur, surtout sur des réseaux lents ou avec des applications très interactives.[11]

II.12 CONCLUSION PARTIELLE

L'architecture client-serveur est un modèle fondamental et durable de l'informatique moderne et distribuée. Sa capacité à séparer les préoccupations entre la présentation (client), la logique métier (serveur d'application) et les données (serveur de données) a permis la création de systèmes hautement évolutifs, sécurisés et maintenables.

Bien que les architectures 2-tiers aient posé les bases, l'évolution vers les modèles 3-tiers et N-tiers a répondu aux défis croissants de scalabilité, de performance et de complexité des applications d'entreprise et du web. L'émergence des services web et des microservices, qui s'inscrivent parfaitement dans le paradigme N-tiers, témoigne de la robustesse et de la flexibilité intrinsèques à cette approche architecturale.

Malgré des inconvénients tels que la complexité de déploiement ou la dépendance au réseau, les avantages (centralisation, sécurité, partage de ressources) surpassent largement les inconvénients dans la grande majorité des cas d'usage contemporains.

CHAPITRE III : SERVICE WEB

III.1 INTRODUCTION PARTIELLE

L'écosystème numérique contemporain est caractérisé par une interconnectivité croissante et une prolifération de systèmes informatiques hétérogènes. Dans ce contexte, la capacité des applications à communiquer et à échanger des informations de manière transparente est devenue non seulement souhaitable, mais impérative. C'est ici que les Web Services entrent en jeu, agissant comme des facilitateurs essentiels de cette communication.

III.2 DEFINITION

❖ *Un écosystème numérique* : Un écosystème numérique est un réseau dynamique d'entités interconnectées (dispositifs IoT, applications cloud, plateformes de données, utilisateurs) interagissant via des interfaces standardisées. Les Web Services en constituent le système nerveux central, permettant la circulation fluide de données hétérogènes. [12]

❖ *Un Web Service* : Un Web Service peut être défini comme une méthode standardisée de communication entre deux applications logicielles différentes, généralement sur un réseau IP, en utilisant des protocoles web. Concrètement, un Web Service agit comme une interface programmatique : il offre une collection d'opérations qui peuvent être invoquées par d'autres applications.

Son rôle principal est de permettre à des systèmes développés avec des technologies différentes (langages de programmation, systèmes d'exploitation, plateformes) de dialoguer et d'interagir sans nécessiter de modifications profondes de leur code ou de leur infrastructure sous-jacente. Il expose des fonctionnalités métier ou des données spécifiques de manière structurée et accessible à distance.

❖ L'objectif principal est de permettre l'interopérabilité entre des systèmes hétérogènes, c'est-à-dire développés dans des langages de programmation ou sur des plateformes distincts (Java, .NET, Python, PHP, etc.). [13]

III.3 CONTEXTE HISTORIQUE

Avant les années 2000, les systèmes informatiques communiquaient principalement via des solutions propriétaires ou des échanges de fichiers batch, ce qui rendait l'intégration complexe, lente et peu flexible. Chaque système avait ses propres formats et protocoles, rendant la compatibilité difficile.

Avec la montée en puissance d'Internet à la fin des années 1990, un besoin de communication en temps réel et standardisée s'est imposé, notamment pour les entreprises cherchant à connecter leurs applications entre elles ou avec des partenaires extérieurs.

C'est dans ce contexte que les Web Services ont émergé. Ils utilisent le protocole HTTP pour le transport des messages et XML pour structurer les données. Cette combinaison a permis de rendre les communications inter-applications universelles, standardisées et indépendantes des technologies utilisées de part et d'autre.

Les Web Services ont marqué une évolution majeure vers les architectures orientées services (SOA). Ces architectures permettent de développer des services réutilisables, modulaires et faiblement couplés, facilitant ainsi la flexibilité, la maintenance et l'intégration des systèmes informatiques. [14]

III.4 RÔLE ET IMPORTANCE DES WEB SERVICES

Le rôle des Web Services est multifacette et leur importance ne cesse de croître :

- *Facilitateur d'Intégration (EAI - Enterprise Application Integration) :*

Leur fonction première est de permettre la communication entre des systèmes disparates au sein d'une même organisation (par exemple, un système de gestion des stocks développé en Java qui doit interagir avec une application de facturation en .NET, ou un CRM avec un ERP). Ils brisent les silos technologiques [15].

- *Support des Applications Distribuées :* Ils sont le fondement des architectures logicielles distribuées, où différentes parties d'une application (frontend,

backend, microservices) résident sur des serveurs distincts et collaborent pour fournir une fonctionnalité complète.

- *Permettre l'Interopérabilité* : Au-delà de l'intégration interne, les Web Services permettent à des entreprises différentes de collaborer en échangeant des données de manière standardisée (par exemple, un site de e-commerce utilisant un Web Service d'une banque pour le paiement ou d'un transporteur pour le suivi de colis).

- *Promotion de la Réutilisation et de la Modularité* : Une logique métier complexe peut être encapsulée dans un Web Service et réutilisée par plusieurs applications clientes. Cela favorise la conception modulaire des systèmes, où les grandes applications sont décomposées en composants plus petits et gérables (une approche souvent désignée sous le terme de microservices).

- *Pilier des Applications Web et Mobiles Modernes* : Pour les applications web riches (Single Page Applications - SPA) et les applications mobiles, les Web Services (particulièrement RESTful) sont indispensables. Ils fournissent l'API (Application Programming Interface) qui permet au frontend (HTML/CSS/JavaScript pour le web, ou code natif pour mobile) de dialoguer avec le backend, d'accéder aux données et d'exécuter la logique métier côté serveur.

III.5 CARACTERISTIQUES FONDAMENTALES DES WEB SERVICES

Plusieurs caractéristiques clés définissent l'essence des Web Services et justifient leur adoption massive :

- *Interopérabilité* : C'est la caractéristique la plus centrale. Les Web Services permettent à des applications écrites dans des langages différents et exécutées sur des systèmes d'exploitation distincts de communiquer sans friction. Ils agissent comme un langage commun, un "traducteur universel" [16].

- *Standardisation* : Ils s'appuient sur des standards ouverts et reconnus, principalement HTTP pour le transport, et XML ou JSON pour le formatage des

messages. Cette adhésion aux standards facilite l'intégration et la consommation des services par un large éventail de clients.

- **Découplage** : Les Web Services favorisent un couplage faible entre le fournisseur (celui qui expose le service) et le consommateur (celui qui utilise le service). Les modifications internes à l'implémentation d'un service n'affectent pas les clients, tant que le contrat de service (l'interface) reste stable. Cela améliore la flexibilité et la maintenabilité des architectures.

- **Réutilisation** : Un Web Service conçu pour une fonctionnalité spécifique peut être exposé et réutilisé par de multiples applications clientes. Cela optimise le développement, réduit la redondance du code et assure une cohérence des logiques métier à travers les systèmes.

- **Accessibilité** : Généralement accessibles via des adresses URL, les Web Services sont faciles à localiser et à invoquer à distance, ce qui est fondamental pour les applications distribuées et les plateformes modernes (web, mobile, IoT).

III.6 TYPES DE WEB SERVICES ET LEUR FONCTIONNEMENT

L'architecture des Web Services se manifeste principalement à travers deux styles distincts : SOAP (Simple Object Access Protocol) et REST (Representational State Transfer). Comprendre leur fonctionnement est essentiel pour choisir l'approche appropriée à chaque contexte.

III.6.1 SOAP (Simple Object Access Protocol)

SOAP est un protocole basé sur XML pour l'échange de messages structurés dans le cadre des Web Services. Il est plus ancien, plus formel et a été largement adopté dans les architectures orientées services (SOA) des grandes entreprises, notamment pour des applications nécessitant un haut niveau de robustesse, de sécurité et de transactionnalité.

III.6.1.A Fonctionnement

Le fonctionnement de SOAP est centré sur l'échange de messages XML. Chaque message SOAP est un document XML bien formé, structuré selon une enveloppe SOAP qui contient :

- *Un en-tête (Header) facultatif* : Il permet d'inclure des informations de contrôle telles que des données d'authentification (pour WS-Security), des informations de routage, ou des données de transaction.
- *Un corps (Body) obligatoire* : Il contient la charge utile du message, c'est-à-dire les données spécifiques à l'opération que le client souhaite effectuer ou la réponse du service .

Le protocole SOAP est indépendant du transport, ce qui signifie qu'il peut utiliser divers protocoles pour acheminer ses messages. Cependant, HTTP et HTTPS sont les plus couramment utilisés en raison de leur ubiquité sur Internet. Les messages SOAP sont souvent envoyés via des requêtes HTTP POST.

Le service SOAP est généralement décrit par un document WSDL (Web Services Description Language). Le WSDL est un fichier XML qui sert de contrat formel pour le service. Il décrit précisément :

- Les opérations que le service expose (par exemple, ``getProduit``, ``ajouterClient``).
- Les messages (requêtes et réponses) associés à chaque opération, y compris les types de données des paramètres.
- Les points d'accès (endpoints), c'est-à-dire les URL où le service peut être atteint [5].

Lorsqu'un client souhaite interagir avec un service SOAP, il :

1. Lit le fichier WSDL pour comprendre les opérations disponibles et la structure des messages.
2. Construit un message SOAP (requête XML) conforme au WSDL.

3. Envoie ce message SOAP via HTTP (généralement POST) à l'URL du service.
4. Le serveur SOAP reçoit le message, le parse (analyse le XML), exécute la logique métier associée à l'opération demandée.
5. Construit un message SOAP de réponse (XML) et le renvoie au client.
6. Le client reçoit et parse la réponse XML pour en extraire les données.

III.6.1.B Diagramme séquence

Participant C as Client (Consommateur)

participant S as Serveur SOAP (Fournisseur)

participant D as Base de Données

C->>S: 1. Demande de WSDL (facultatif, pour initialisation)

S-->>C: 2. Envoie WSDL (description du service)

C->>S: 3. Requête SOAP (message XML via HTTP POST)

S->>S: 4. Parse le message XML

S->>D: 5. Accède/Modifie Données (Logique Métier)

D-->>S: 6. Retourne Données

S->>S: 7. Construit la réponse SOAP (message XML)

S-->>C: 8. Réponse SOAP (message XML via HTTP)

C->>C: 9. Parse la réponse XML et utilise les données

III.6.1.C Avantages Et Inconvénients

Tableau 1 Avantages et Inconvénients du SOAP

AVANTAGES	INCONVÉNIENTS
Robustesse et Fiabilité : Supporte des transactions complexes (ACID), des messages fiables et des spécifications de sécurité avancées (WS-Security).	Complexité : Le protocole est verbeux (XML) et les spécifications WS-* sont nombreuses et complexes à maîtriser.
Indépendance du Transport : Peut utiliser	Performance : La surcharge due au XML

HTTP, SMTP, JMS, etc., offrant une flexibilité pour des scénarios variés.	(taille des messages, parsing) peut impacter la performance, notamment sur des réseaux à faible bande passante ou pour des applications à haute fréquence.
Forte Standardisation : Le WSDL fournit un contrat formel qui facilite la découverte et l'intégration automatique (génération de code client).	Courbe d'apprentissage Abrupte : Nécessite des outils et frameworks spécifiques qui peuvent être lourds et complexes à configurer.
Sécurité Avancée : Les extensions WS-Security offrent des mécanismes de sécurité granulaires au niveau du message.	Moins adapté aux applications légères comme les mobiles ou l'IoT en raison de sa verbosité et de sa complexité.

III.6.2 REST (Representational State Transfer)

REST n'est pas un protocole, mais un style architectural qui a été formalisé par Roy Fielding en 2000 [17]. Il s'est imposé comme l'approche dominante pour la conception d'APIs Web grâce à sa simplicité, sa légèreté et son alignement avec les principes fondamentaux du World Wide Web.

III.6.2.A Principes Architecturaux des Services RESTful

Un service est dit "RESTful" s'il adhère à un ensemble de contraintes architecturales clés :

1. **Client-Serveur** : Séparation stricte des préoccupations entre l'interface utilisateur (client) et le stockage des données/logique métier (serveur). Cette séparation améliore la portabilité de l'interface et la scalabilité du serveur.

2. **Stateless (Sans État)** : Chaque requête du client vers le serveur doit contenir toutes les informations nécessaires pour que le serveur la comprenne et la traite. Le serveur ne doit pas stocker d'informations de session entre les requêtes. Cela garantit

que chaque requête est indépendante, facilitant la scalabilité horizontale et la tolérance aux pannes [45].

3. *Cachable* : Les réponses des requêtes (en particulier GET) peuvent être cachées pour améliorer la performance et la scalabilité en réduisant la charge serveur.

4. *Système de Couches (Layered System)* : Le client n'a pas besoin de savoir s'il est directement connecté au serveur final ou à un intermédiaire (proxy, load balancer). Cela permet d'ajouter des couches d'infrastructure sans affecter le client.

5. *Interface Uniforme* : C'est la contrainte la plus importante et la plus distinctive de REST. Elle est composée de quatre sous-contraintes :

- Identification des ressources par des URI : Chaque élément clé du système est une ressource (ex: un utilisateur, un message, une délibération) identifiable de manière unique par une URI (Uniform Resource Identifier), qui est l'adresse de la ressource (par exemple, `/api/users/123`).

- Manipulation des ressources par des représentations : Les clients interagissent avec les ressources en manipulant leurs représentations. Les données sont échangées dans des formats standard comme JSON (JavaScript Object Notation) ou XML (eXtensible Markup Language). JSON est le format prédominant pour sa légèreté et sa facilité d'utilisation avec JavaScript [46].

- Messages auto-descriptifs : Chaque message contient suffisamment d'informations pour que le récepteur puisse le comprendre, y compris le type de média de la représentation (Content-Type).

- HATEOAS (Hypermedia As The Engine Of Application State) : Les représentations des ressources contiennent des liens (hypermedia) qui permettent au client de découvrir les actions disponibles et l'état suivant de l'application. C'est un principe avancé de REST souvent moins strictement appliqué dans les APIs courantes.

III.6.2.B Fonctionnement

Un service RESTful s'appuie directement sur le protocole HTTP et ses méthodes standard pour les opérations sur les ressources :

- ❖ GET : Pour récupérer une ressource ou une collection de ressources (lecture).
- ❖ POST : Pour créer une nouvelle ressource.
- ❖ PUT : Pour mettre à jour une ressource existante (remplacement complet).
- ❖ DELETE : Pour supprimer une ressource.
- ❖ PATCH : Pour mettre à jour partiellement une ressource (moins commun mais utile).

III.6.2.C Le processus d'interaction :

1. Le client formule une requête HTTP (avec la méthode appropriée et l'URI de la ressource).
2. Le client peut inclure des données dans le corps de la requête (pour POST/PUT/PATCH), généralement au format JSON.
3. Le serveur reçoit la requête, identifie la ressource et l'opération demandée.
4. Il exécute la logique métier correspondante (par exemple, interagit avec une base de données).
5. Le serveur construit une réponse HTTP, incluant :
 - Un code de statut HTTP (ex : 200 OK, 201 Created, 404 Not Found, 500 Internal Server Error) pour indiquer le résultat de l'opération.
 - Des en-têtes HTTP (ex : `Content-Type: application/json`).
 - Un corps de réponse, souvent une représentation de la ressource au format JSON.

6. Le client reçoit la réponse, analyse le code de statut et le corps JSON pour utiliser les données.

III.6.2.D Graph LR

C[Client (Navigateur/Mobile)] -->|GET /utilisateurs| S[Serveur API REST]

S -->|200 OK, JSON des utilisateurs| C

C -->|POST /utilisateurs (JSON du nouvel utilisateur) | S

S -->|201 Created, JSON du nouvel utilisateur| C

C -->|PUT /utilisateurs/123 (JSON mise à jour)| S

S -->|200 OK| C

C -->|DELETE /utilisateurs/123| S

S -->|204 No Content| C

III.6.2.E Avantages et Inconvénients

Tableau 2 *Avantages et Inconvénients du RestFul*

AVANTAGES	INCONVÉNIENTS
Légèreté et Performance : Utilisation de JSON et HTTP rend les communications rapides et efficaces.	Moins de Standardisation Formelle : Pas de WSDL équivalent, ce qui peut rendre la documentation et la découverte plus informelles (mais des outils comme OpenAPI/Swagger combleront cette lacune).
Simplicité : Facile à comprendre et à implémenter, car il exploite directement les conventions du Web.	Sécurité à Implémenter Manuellement : Les mécanismes de sécurité avancés (authentification, autorisation) doivent être ajoutés via des jetons (JWT, OAuth 2.0), ils ne sont pas "intégrés" comme avec WS-Security.
Scalabilité : La nature stateless et l'utilisation de HTTP facilitent la mise à	Limité pour les Opérations Complexes : Moins adapté aux besoins de transactionnalité

l'échelle horizontale.	distribuée ou de fiabilité des messages garanties par certaines spécifications SOAP.
Flexibilité : Peut échanger des données dans différents formats (JSON, XML, texte).	Adapté aux Applications Modernes : Idéal pour les applications web (SPA), mobiles et IoT.

III.7 ARCHITECTURE DE COMMUNICATION ET PRINCIPES GENERAUX

Au-delà des styles spécifiques (SOAP ou REST), les Web Services s'inscrivent dans des architectures de communication qui définissent comment les différents composants interagissent et gèrent l'état.

III.7.1 Modèle Client-Serveur

Le modèle Client-Serveur est le paradigme fondamental sur lequel reposent les Web Services. Le client (l'application qui consomme le service) initie la communication en envoyant des requêtes, et le serveur (l'application qui fournit le service) répond à ces requêtes. Cette séparation des rôles et des responsabilités améliore la modularité, la gestion des ressources et la maintenance .

III.7.1.A Synchronisme et Asynchronisme des Communications

La manière dont les requêtes et les réponses sont gérées dans le temps est un aspect clé :

- *Communication Synchrone* : C'est le modèle le plus répandu pour les Web Services. Le client envoie une requête et attend une réponse immédiate du service avant de pouvoir poursuivre d'autres opérations. Si le service met du temps à répondre, le client peut être bloqué. C'est le comportement par défaut des appels RESTful et de nombreuses opérations SOAP [16].

- *Communication Asynchrone* : Dans ce modèle, le client envoie une requête et ne bloque pas son exécution en attendant la réponse. La réponse sera reçue plus tard, souvent via un mécanisme de rappel, une file d'attente de messages ou une notification.

Ce modèle est crucial pour des opérations longues, pour éviter le blocage du client ou pour des architectures où la fiabilité de la livraison des messages est primordiale (ex : systèmes bancaires, streaming de données) . Bien que les Web Services eux-mêmes soient majoritairement synchrones, ils peuvent s'intégrer dans des architectures asynchrones en utilisant des bus de messages ou des files d'attente.

III.7.1.A Gestion de l'État : Stateless vs. Stateful

La manière dont l'état d'une interaction est géré entre le client et le serveur est une distinction cruciale :

- *Stateless (Sans État)* : Chaque requête est complètement indépendante des requêtes précédentes. Le serveur ne conserve aucune information sur l'état de la session client entre les requêtes. Toutes les données nécessaires pour traiter une requête doivent être incluses dans la requête elle-même. C'est un principe fondamental de REST qui contribue massivement à la scalabilité, car n'importe quel serveur peut traiter n'importe quelle requête à tout moment, et la défaillance d'un serveur n'entraîne pas la perte de l'état de la session client .

- *Stateful (Avec État)* : Le serveur conserve des informations sur les interactions précédentes du client, établissant ainsi une "session" entre le client et le serveur. Bien que cela puisse simplifier certaines logiques métier complexes, cela rend la scalabilité plus difficile (le client doit toujours interroger le même serveur) et augmente la complexité en cas de panne du serveur .

III.8 CONCLUSION PARTIELLE

Les Web Services sont devenus un pilier incontournable de l'architecture logicielle contemporaine. En définissant des moyens standardisés et interopérables pour la communication entre applications, ils ont permis de transformer des systèmes monolithiques en architectures distribuées, plus flexibles, modulaires et scalables. Qu'il s'agisse du protocole formel SOAP, privilégié pour sa robustesse en entreprise, ou du

style architectural léger et populaire REST, les Web Services sont au cœur des interactions entre les composants logiciels.

Leur rôle essentiel dans l'intégration de systèmes hétérogènes, la réutilisation de la logique métier et le support des applications web et mobiles modernes, justifie leur omniprésence

CHAPITRE IV : ETUDE PREALABLE ET MODELISATION

IV.1 INTRODUCTION PARTIELLE

Dans ce chapitre, une étude est faite du système existant, une analyse permettant d'examiner ce système, de déterminer ses points forts et ses défaillances ainsi qu'analyser les nouveaux besoins. En fonction du critique de l'existant, on va déceler le fond du problème est de trouver une solution adéquate pour y remédier, et ensuite passer à la modélisation du nouveau système qui doit être exécuté.

IV.2 ETUDE PREALABLE

IV.2.1 PRESENTATION DE L'UNIVERSITE OFFICIELLE DE MBUJIMAYI

IV.2.1.A Aperçu Historique

L'Université Officielle de Mbuji-Mayi (UOM en sigle) est une Institution Publique d'Enseignement Supérieur et Universitaire reconnue par l'État Congolais.

L'UOM est le fruit d'une métamorphose de ce qui a été Centre Universitaire de Mbuji-Mayi/Université d'État (CUM/UE), le CUM/UE est une extension de l'Université de LUBUMBASHI, avant d'acquérir le statut d'une Université Autonome Officielle au même rang que celles de LUBUMBASHI (UNILU), KINSHASA (UNIKIN), KISANGANI (UNIKIS), etc.

Créé officiellement en 1993, le C.U.M/U. En 'a ouvert ses portes que l'année académique en 1995-1996. A ses débuts, le C.U.M/U. En avait pas de site propre. C'est ainsi que les étudiants étaient nomades, obligés de temps en temps de trouver abris dans les Institutions d'Enseignement Primaire, Secondaire et Professionnelle (E.P.S.P).

Après avoir occupé plusieurs sites, l'institut et allée à l'école BUTAMINA vu le nombre des étudiants, avant de s'installer finalement dans les deux bâtiments qui sont le site actuel de l'UOM depuis 1996.

Lorsqu'il y a eu un projet de création des pré-Universités sur l'ensemble du pays, c'est ainsi l'Athénée de Mbuji-Mayi fut choisi.

Le C.U.M/U.E est devenu une extension de l'UNILU en 2001 par l'arrêté Ministériel N°MINEDUC/CAB.MIN/086, du 03 Septembre 2001. En 2003 il sera

précédé du Professeur KABONGO MAKANDA (DG), Professeur KAZADI MUKENGE (SA) et le chef de travaux KAPINGA DIBENDELE (SA et financier).

La vocation universitaire a été consacrée par l'Arrêté Ministériel N°MINESU/CAB.MIN/RS du 05 Octobre 2005 portant l'autonomie de l'extension, par cet acte Juridique, le Gouvernement Congolais, à travers son Ministre de l'ESU, avait décidé d'élever le C.U.M/U. Eaurang d'une Université Officielle à l'instar de toutes celles qui existent dans le pays. L'appellation retenue a été « Université Officielle de Mbujimayi (UOM en sigle) ».

IV.2.1.B Organes De L'U.O.M

L'U.O.M comprend les organes ci-après :

- Conseil de l'Université qui est un organe de conception, d'orientation et de suivi des politiques tracées ; il est composé de toutes les forces vives de l'Université, à savoir le Comité de Gestion, le Délégué du Corps Académique, le Délégué du Corps Scientifique, le Délégué du Personnel Administratif, Technique Ouvrier ainsi que le Délégué de la Communauté estudiantine.
- Le Comité de Gestion est composé d'un Recteur, d'un Secrétaire Général Académique, d'un Secrétaire Général Administratif et d'un Administrateur de Budget.

IV.2.1.C Les Facultés.

Les Facultés Organisées à L'UOM sont :

1. Faculté de Médecine ;
2. Faculté de Droit ;
3. Faculté des Sciences Économiques et de Gestion ;
4. Faculté des Sciences Agronomiques à partir de 1998 ;
5. Faculté des Sciences Sociales, Administratives Politiques :
 - Département des Sciences Politiques administratives ;
 - Département des Relations Internationales :2005 à nos jours ;

–Département des Sciences de l'Information et de la Communication : 2007 à nos jours.

6. Faculté des Sciences :

- Département de Géologie : 2007 à nos jours ;
- Département de Chimie :1996 à 1999 ;
- Département de l'Informatique : 2017 à nos jours.
- Département de polytechnique : 2021 à nos jours

IV.2.1.D Situation Geographique

L'UOM est situé au N°06, sur l'avenue DILUNGA, au quartier BUBANJI, dans la commune de la DIULU, Ville de MBUJIMAYI, dans la province du Kasai Oriental, en République Démocratique du Congo

IV.2.1.E Objectifs De L'institution Dans La Societe

A l'instar des autres Universités, l'UOM est une Institution Publique, ouverte à tous sans discrimination, pour suit une triple mission à savoir l'Enseignement, la Recherche et le Service de la Nation.

IV.3 DIAGNOSTIC DU PREABLE

Après l'analyse de la structure de l'Université et ses diverses informations qui y circulent, nous passons à l'établissement du diagnostic de l'existant.

IV.3.1 RECENSEMENT DES PROBLEMES

POINT FORT :

Les normes de délibération académiques ont été bien élaboré au niveau du ministère de l'enseignement supérieur et universitaire en RDC.

POINTS FAIBLES :

Étant donné que L'Université Officielle de Mbujimayi a pour vocation de répondre aux exigences des nouvelles technologies de l'information et de la communication, aucune modélisation de communications n'a été définie encore moins pour le partage et circulation de l'information.

L'Université Officielle de Mbujimayi ne propose pas de modélisation d'une infrastructure informatique devant répondre aux besoins des institutions dans le cadre de la circulation de l'information entre les différents secteurs.

Dans ce contexte, l'informatique devenant un outil incontournable matière de gestion, L'Université Officielle de Mbujimayi ne prévoit pas un poste d'un responsable (directeur) informatique ainsi que ses subalternes en vue de s'occuper de toutes les questions inhérentes à l'organisation informatique au sein de l'établissement.

L'enseignement est actuellement devenu un acteur important dans le système d'information de l'UOM. Malheureusement mis à l'égard dans la gestion d'échange

A la lumière des imperfections qui viennent d'être reprises dans le système existant, nous pouvons proposer quelques solutions dans le but d'améliorer la performance et le bon fonctionnement du système existant.

IV.3.2 ETUDE DES POSTES DE TRAVAIL

➤ **Apparitaire :** Ce poste de travail, se charge de l'élaboration du programme de cours et de la communication de ce dernier.

➤ **Le bureau du jury :** Ce poste de travail, se charge de la récolte de tous les points des étudiants (travail journalier (TJ) et Examen) afin de pouvoir faire le calcul possible (délibérer) et communiquer les résultats aux étudiants.

➤ **Etudiant :** Est la personne principale qui est là pour consulter le programme de cours et ses résultats de la délibération.

➤ **Le Webmaster(Administrateur) :** C'est lui qui gère les comptes des utilisateurs, insère les résultats de la délibération pour le compte de chaque étudiants et contrôle l'ensemble du système.

IV.3.3 CRITIQUE DE L'EXISTANT

L'université officielle de Mbuji-Mayi ne possède aucun système préétabli qui gère ni la délibération ou la communication de programme. Ce qui fait à ce que tout le processus de délibération et de communication de cours soit manuel entraînant ainsi le retard et les erreurs parfois une grande difficulté d'atteindre le maximum de la communauté. C'est à partir de ces causes que seront élaborées les solutions futures.

IV.3.4 PROPOSITION DES SOLUTIONS ET ORIENTATIONS

Au regard des observations énumérées ci - dessus, dans cette phase, il s'agit de proposer des idées de solutions fonctionnelles pour aboutir à un nouveau système qui répondra aux difficultés rencontrées.

Ainsi comme solution générale nous avons pensé à mettre en place une plateforme web qui permettra à la couverture de toutes les activités.

IV.4 MODELISATION DU NOUVEAU SYSTEME

La modélisation est une activité fondamentale de conception des systèmes d'information, consistant à élaborer des représentations abstraites (modèles) pour analyser un système existant ou concevoir un nouveau système. Comme le souligne Dennis (2015), "Un modèle est une simplification intentionnelle de la réalité, conçue pour faciliter la compréhension des structures complexes en éliminant les détails non essentiels"[18]. Cette phase cruciale établit les fondations de l'application en traduisant les exigences métier en structures formelles.

IV.4.1 STRUCTURE GENERALE DE LA SOLUTION PROPOSEE

Notre solution consistera à créer une application web purement mobile enfin de permettre aux étudiants de consulter les programmes de cours et résultats de la délibération, Echanger les Idées avec l'ensemble de la communauté estudiantine grâce à la fil d'actualité et la messagerie et permettra aussi au Bureau du jury de publier d'une manière simple et facile le résultat de la délibération.

IV.4.2 L'ETUDE CONCEPTUELLE DU NOUVEAU SYSTEME

IV.4.2.A Règles du processus :

Ces règles expriment la façon auxquelles obéit le futur système d'information. Pour cela, nous appliquerons l'ensemble des règles de vérification et de normalisation suivantes :

- Un compte étudiant peut-être créer que s'il a un matricule
- Un compte étudiant peut voir ou faire les publications
- Un compte étudiant peut envoyer ou recevoir les messages
- Un compte étudiant peut consulter ses résultats de la délibération
- Un administrateur peut ajouter un nouveau matricule
- Un administrateur peut ajouter les résultats de la délibération pour les comptes des étudiants.

IV.4.2.B Schéma de circulation d'informations

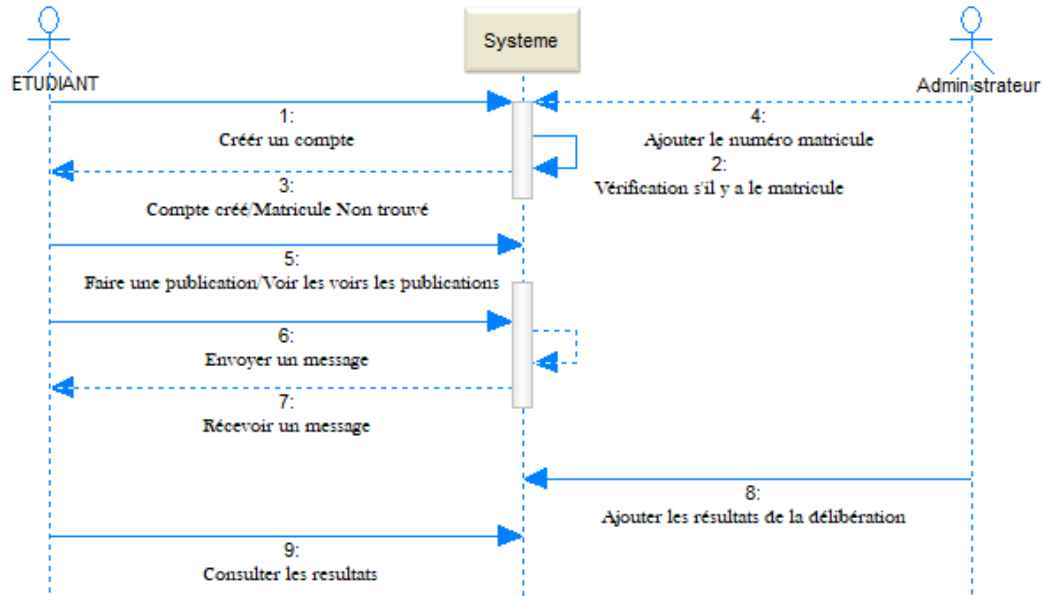


Figure 5 Sch  ma de circulation

IV.4.3 PR  SENTATION DES M  THODES

Notre d  marche s'appuie sur MERISE, m  thodologie structur  e organis  e en trois cycles :

1. Cycle d'abstraction

- Niveau conceptuel : D  finition des r  gles de gestion et finalit  s m  tier (ind  pendant de la technique)
- Niveau organisationnel : Organisation des processus et donn  es ("qui fait quoi, o   ?")
- Niveau physique : Impl  mentation technique ("comment ?")

2. Cycle de vie

- Conception : Analyse de l'existant et mod  lisation du syst  me
- R  alisation : D  veloppement et d  ploiement
- Maintenance : Adaptation aux   volutions futures

3. Cycle de d  cision

- Prise de d  cisions graduelles tout au long du projet

IV.4.3.A Modèle Conceptuel Des Données (MCD)

Le MCD représente le cœur de la modélisation MERISE, décrivant les concepts métier indépendamment des implémentations techniques.

❖ *Concepts Fondamentaux*

1. *Entité* : Objet du monde réel (concret ou abstrait) possédant une existence autonome et caractérisé par des propriétés homogènes.

Exemple : `Étudiant`, `Résultat` [19]

2. *Identifiant* : Attribut unique permettant d'identifier sans ambiguïté chaque occurrence d'une entité.

Exemple : `Matricule` pour `Étudiant`

3. *Attribut* : Information élémentaire caractérisant une entité ou une relation, conforme aux règles métier.

Exemple : `Nom`, `Date\_Naissance` pour `Étudiant`

4. *Association* : Lien sémantique significatif entre entités, définissant une interaction métier.

Exemple : `Consulter` entre `Étudiant` et `Résultat`

5. *Cardinalités* : Détermine les contraintes numériques précisant :

- Participation minimale (0 ou 1)
- Participation maximale (1 ou N)

Exemple : Un étudiant consulte (0,N) résultats ; Un résultat appartient à (1,1) étudiant

❖ *Construction Du Modèle Conceptuel De Données (MCD)*

1. Recensement des entités

- Compte Étudiant
- Publication
- Message
- Résultats

2. Les attributs ou propriétés

- Id_compte, nom, postnom, prenom, sexe, âge, code, promotion, faculté, photo
- Id_publication, date_publication, nombre_vues, contenu, photo
- Id_message, Contenu_message
- Id_resultats, Sesson, Moyenne, Mention, Statut.

A. Modèle comptuel de donnée (MCD)

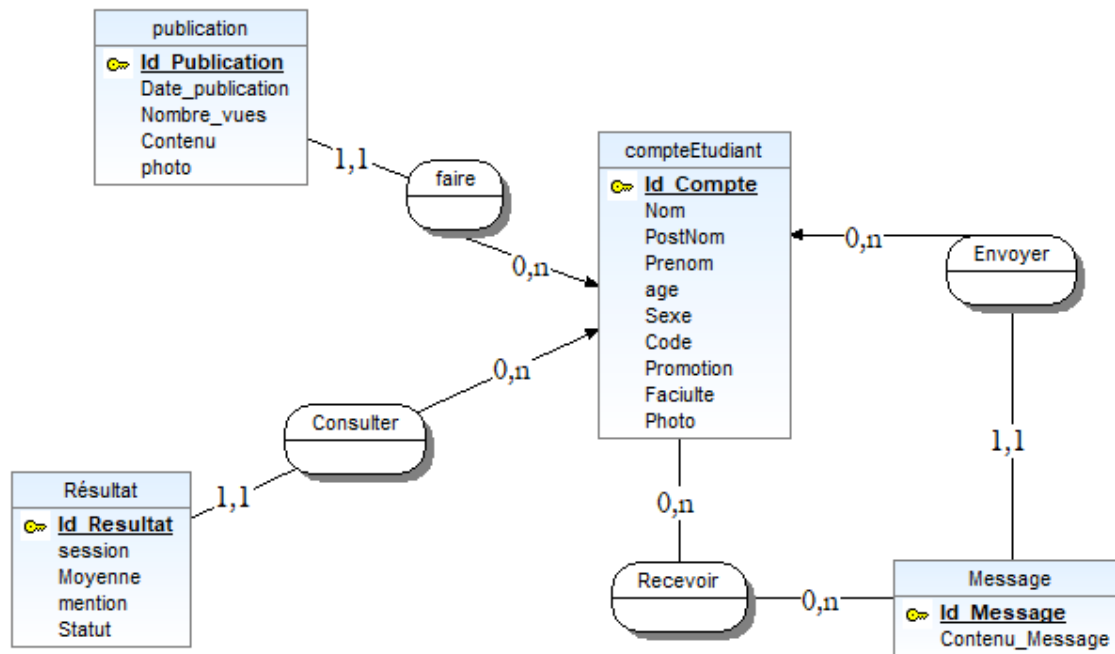


Figure 6 Modele conceptuel de donnée

B. Modèle logique de données (MLD)

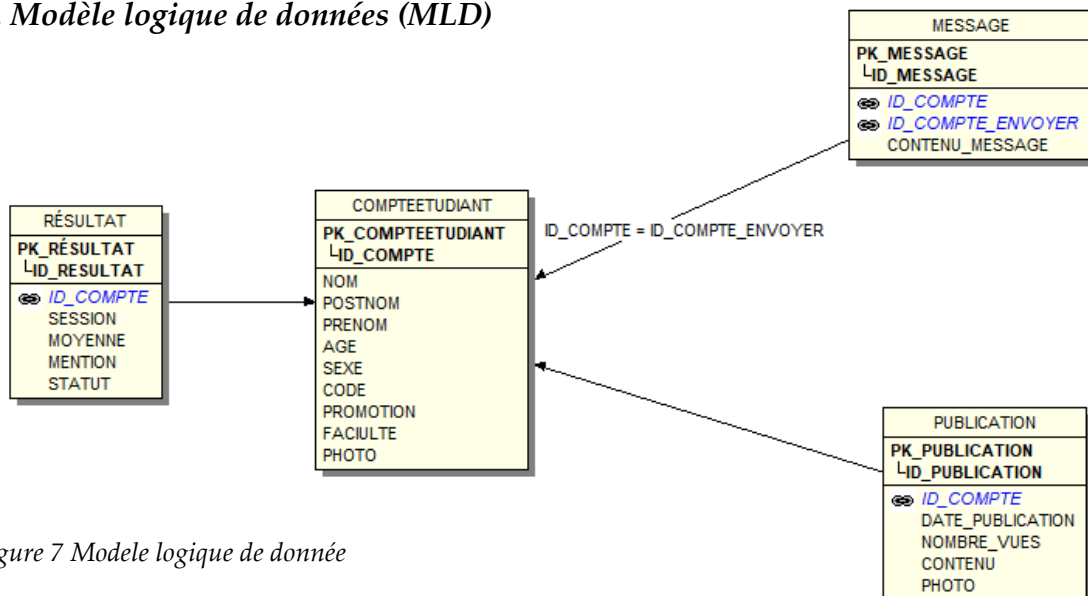


Figure 7 Modele logique de donnée

C. Modèle physique de données (MPD)

Tableau 3 modèle physique de donnée

COMPTE_ETUDIANT			
CHAMPS	TYPE	TAILLE	Observation
Id_compte	Numérique	10	Clé primaire
Nom	Texte	15	
Postnom	Texte	20	
Prenom	Texte	20	
Sexe	Texte	5	
Âge	Numérique	5	
Code	Alphanumérique	20	
Promotion	Alphanumérique	20	
Faculté	Texte	20	
Photo	Blob	20	
Id_matricule	Numérique	50	
PUBLICATION			
CHAMPS	TYPE	TAILLE	Observation
Id_publicatin	Numérique	10	Clé primaire
Date_publication	Date	15	
nombre_vues	Numérique	20	
contenu	Alphanumérique	100	
Photo	Blob	20	
Id_Compte	Numérique	10	

MESSAGE			
CHAMPS	TYPE	TAILLE	Observation
Id_message	Numérique	10	Clé primaire
message	Texte	15	
Id_compte	Numérique	10	
Id_compte_envoyer	Numérique	10	

RESULTATS			
CHAMPS	TYPE	TAILLE	Observation
Id_Résultats	Numérique	10	Clé primaire
Id_compte	Numérique	15	
Moyenne	Numérique	10	
Mention	Numérique	10	
pourcentage_validation	Numérique	10	
Statut	texte	10	

IV.5 CONCLUSION PARTIELLE

Dans ce chapitre nous avons eu à faire une étude préalable du système existant tout en prélevant le point fort comme le respect de la norme académiques et les points faibles comme un manque de système informatisé donc une gestion totalement manuelle après cette étude nous avons eu à proposer les solutions et enfin établie une nouvelle modélisation adaptée à notre champs et problème

CHAPITRE V : DEPLOIEMENT

V.1 INTRODUCTION PARTIELLE

La réalisation d'une plateforme web mobile interactive pour la consultation des résultats de délibération, intégrant messagerie et fil d'actualité, constitue l'objectif central de ce mémoire. Ce cinquième chapitre présente les différentes phases méthodologiques et techniques menant à la concrétisation de l'application.

V.2 IMPLEMENTATION

L'implémentation représente l'étape cruciale où les conceptions théoriques et les modèles abstraits prennent vie sous la forme d'un système fonctionnel. C'est le processus de traduction des spécifications logiques et des diagrammes de conception en code exécutable et en structures de données concrètes. En d'autres termes, l'implémentation est la phase de construction concrète de la solution.

V.2.1 Outils utilisés

Afin d'aboutir à la solution, nous nous sommes servis des outils suivants:

❖ *Technologies Frontend (Interface Utilisateur)*

Pour l'expérience mobile responsive :

- *HTML5* : Structure sémantique des pages
- *CSS3* : Design adaptatif et animations
- *JavaScript* : Interactivité dynamique et gestion des événements
- *Framework Bootstrap 5* : Composants pré-stylisés pour accélérer le

développement

❖ *Technologies Backend (Logique Métier)*

Pour le traitement des données et services :

- *PHP 8* : Langage serveur pour requêtes dynamiques et API
- *MySQL* : Système de gestion de base de données relationnelle

❖ *Environnement Technique*

- *XAMPP* : Pack serveur local intégrant Apache (serveur web), MySQL (SGBD), et PHP
- *phpMyAdmin* : Interface web pour l'administration de la base de données
- *Visual Studio Code* : Éditeur avec extensions pour HTML/CSS/JS/PHP

V.2.2 Justification des choix techniques :

Tableau 4 Justification de choix technique

COMPOSANT	SOLUTION RETENUE	AVANTAGES
Interface	HTML5/CSS3/JS	Compatibilité mobile, légèreté, standards web
Backend	PHP + MySQL	Stabilité, documentation abondante, intégration XAMPP
Modélisation	MERISE	Rigueur dans la structuration des données
SGBD	MySQL (via XAMPP)	Performances, sécurité, gestion via phpMyAdmin

V.2.3 Implémentation de la Base de Données :

Cette partie sera consacrée à la matérialisation de notre modèle de données. Nous démontrerons comment cette structure est directement dérivée de notre modélisation initiale pour garantir l'intégrité et la performance des données.

Comme déjà annoncé précédemment, nous utilisons MySQL qui est intégré dans phpAdmin, ainsi nous avons d'abord débuté par la création de la base de donnée ayant comme Nom « authentification », et c'est là que nous avons pu créer nos 5 tables.

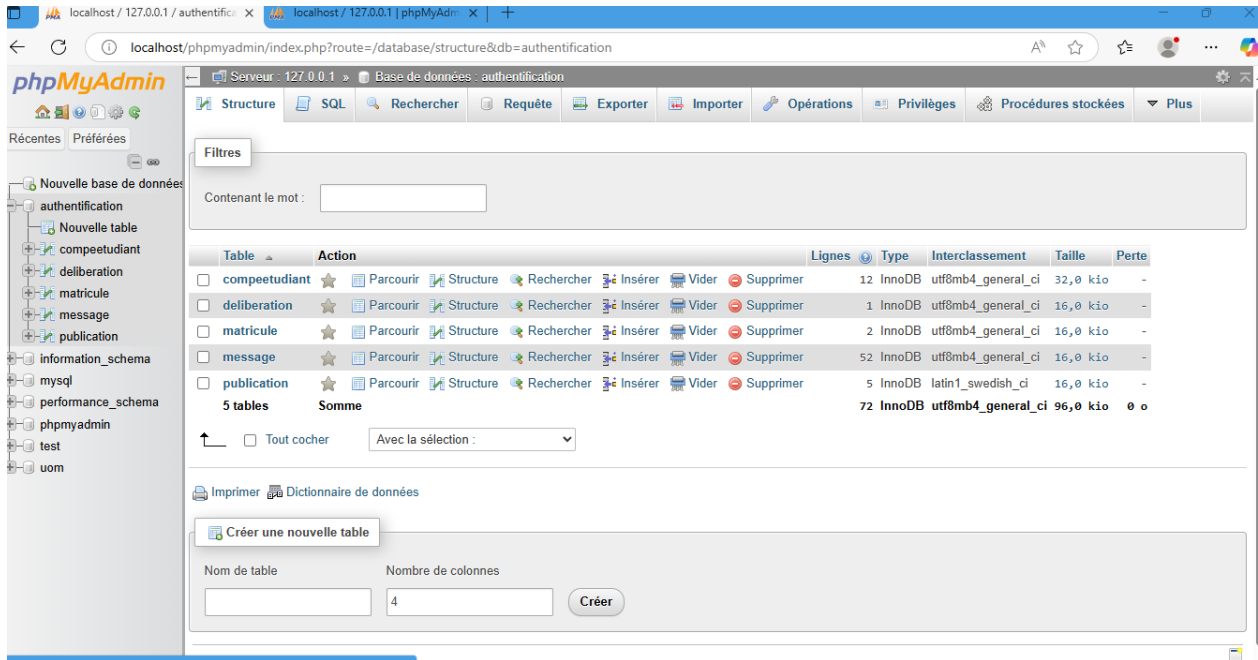


Figure 8 Création de la base de donnée sur phpAdmin

Ici nous avons démontré le processus de conception de la table compte étudiant.

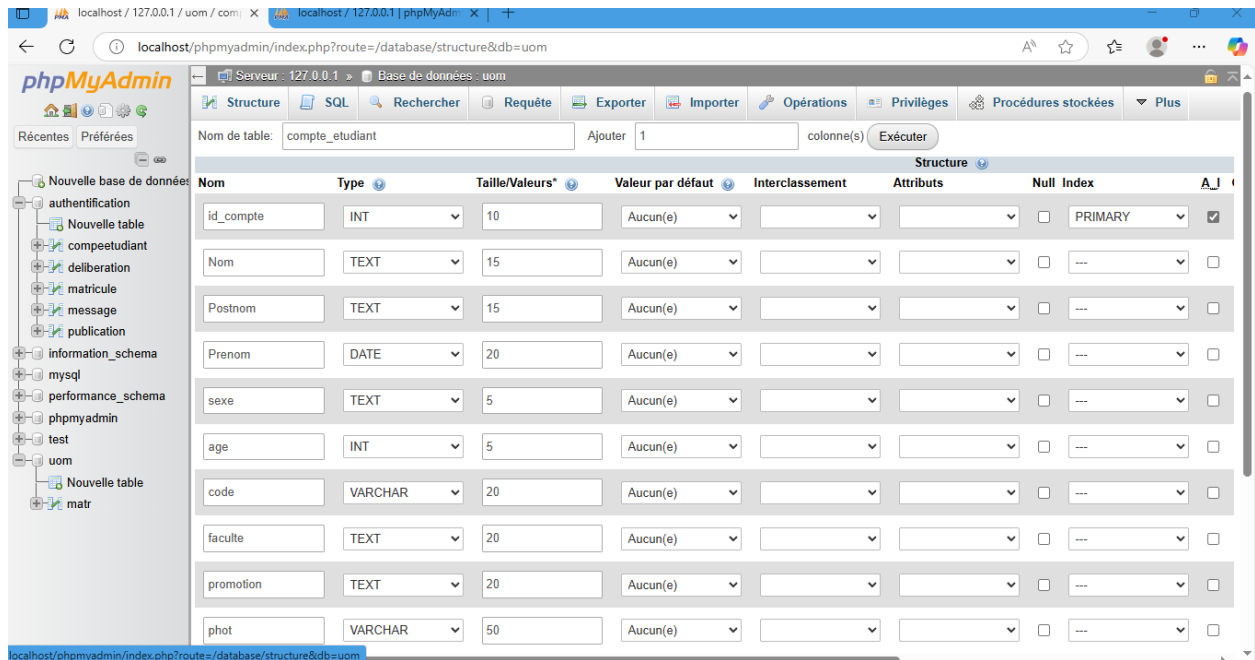


Figure 9 Création de la table Etudiant

Ici nous démontrons le processus relationnel entre la table matricule et compte_etudiant.

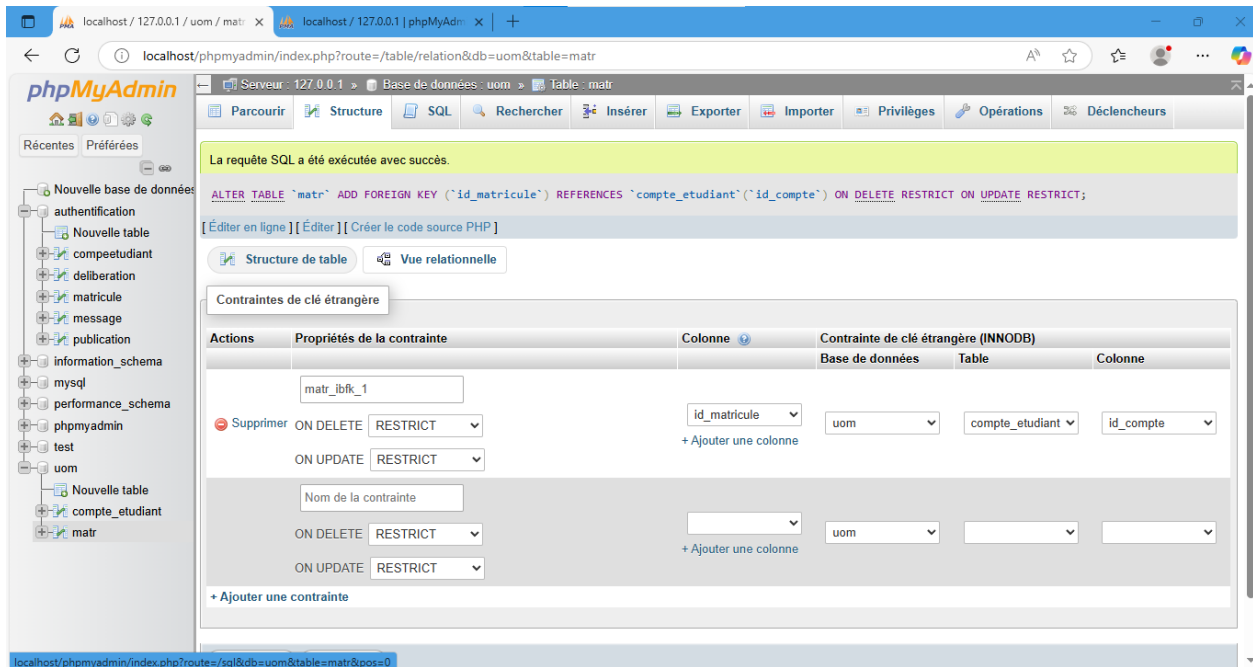


Figure 10 .Création de la relation

V.2 .4 Conception de l'Architecture de l'Application :

Nous y présenterons l'organisation générale de notre solution, la manière dont les différents modules et composants ont été structurés et intégrés pour assurer la fonctionnalité attendue de l'application.

Nous allons maintenant explorer comment les technologies mentionné précédemment (Pour le frontend : Html, css, js Et pour le backend : php et MySQL) ont été agencées pour donner vie à notre solution.

❖ Organisation des Modules et Flux de donnée

Notre application a été conçue selon une architecture client-serveur avec une API REST. Cette approche a permis de diviser le système en modules distincts, chacun ayant une responsabilité claire et des interactions bien définies.

1. *Le Module d'authentification* : Ce module est responsable de la gestion de l'authentification des utilisateurs.

Avant d'accéder à l'authentification nous avons d'abord une page de démarrage qui doit se charger et puis après quelques temps la page d'authentification s'affiche.

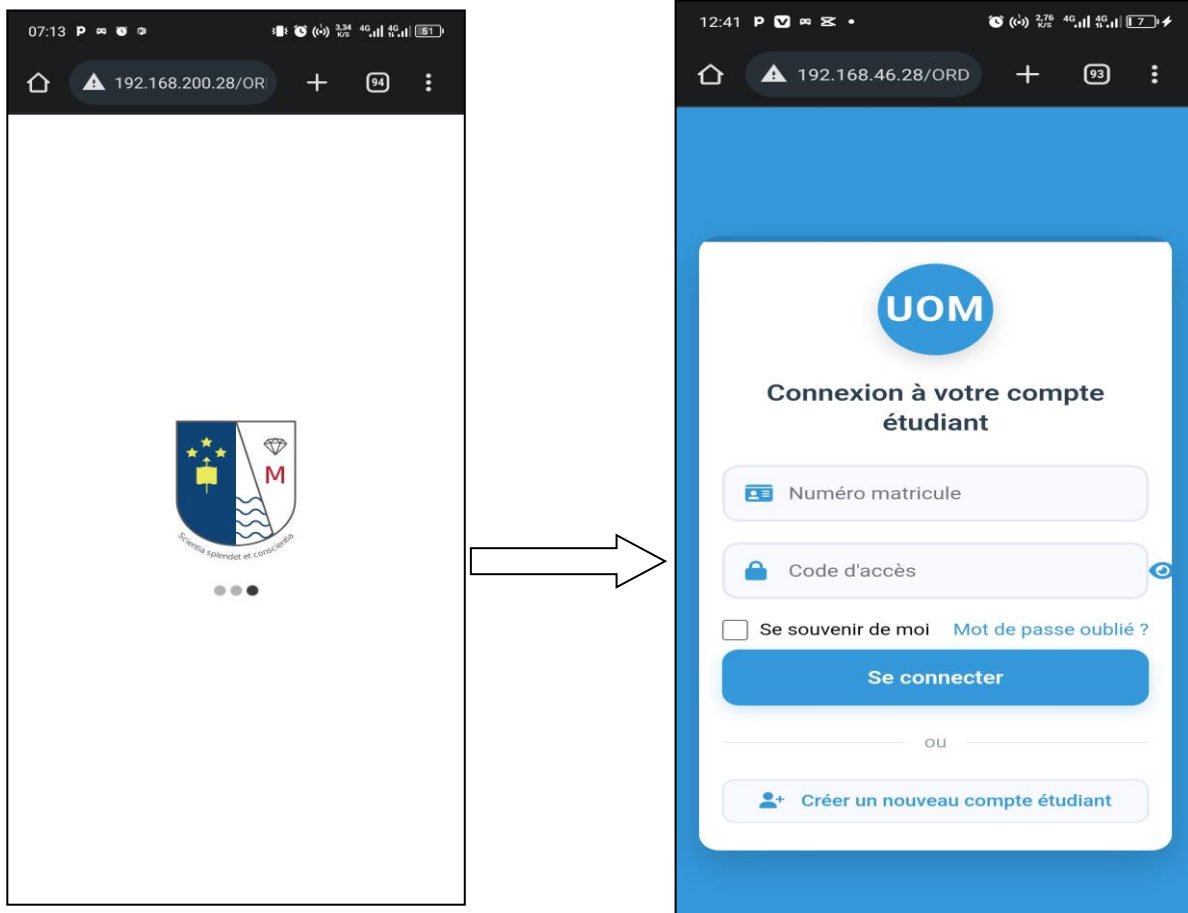


Figure 11 Interface de démarrage et authentification

2. Le Module création du compte : Sa fonction principale est de permettre aux nouveaux utilisateurs de s'enregistrer s'ils ont un matricule propre.

S'il arrivait que l'utilisateur n'a pas encore créer un compte avec son numéro matricule, là il peut alors passer à la page de création du compte.

Et ne peut créer un compte dans cette application que celui qui a un numéro matricule bien enregistré, si l'utilisateur essaie de forcer on lui affichera un message pour lui demander de chercher un matricule valide.

07:30 P 192.168.200.28/OR 94

Votre avenir commence ici

Communauté UOM

Créez votre compte étudiant pour accéder à tous les services académiques et rester connecté

Promotion

Choisissez votre promotion

Faculté

Nom de votre faculté

Code d'accès (Mot de passe)

Votre code est personnel

Photo de reconnaissance (Profil)

Choisir un fichier Aucun...r choisi

Créer le compte

Figure 12 Interface de création du compte

3. *Le Module de la file d'actualité* : Sa fonction principale est de gérer l'affiche de l'actualité et aussi la publication des nouvelles Actualités.

Ce module affiche sa page lorsque l'authentification de l'utilisateur a réussie et que l'utilisateur a même confirmé ces information (Nom, Postnom, Prenom, Photo) .

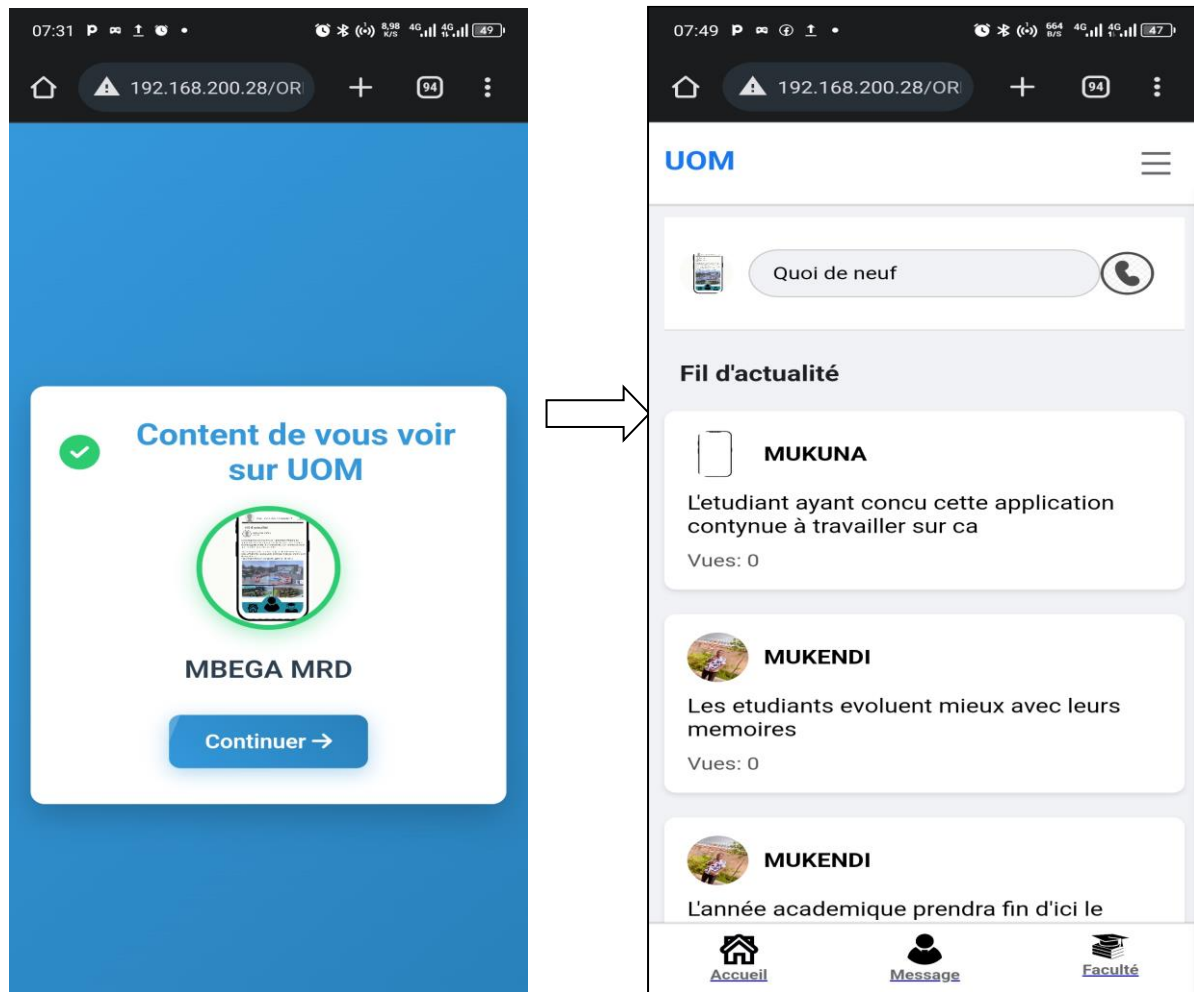


Figure 13 Interface d'accueil et fil d'actualité

4. Le Module de la messagerie : Sa fonction principale est de gérer une communication privée entre 2 utilisateurs.

Là il y'a une liste de tous les contacts, l'utilisateur peut choisir avec qui il veut communiquer ou dialoguer puis lancer la conversation.

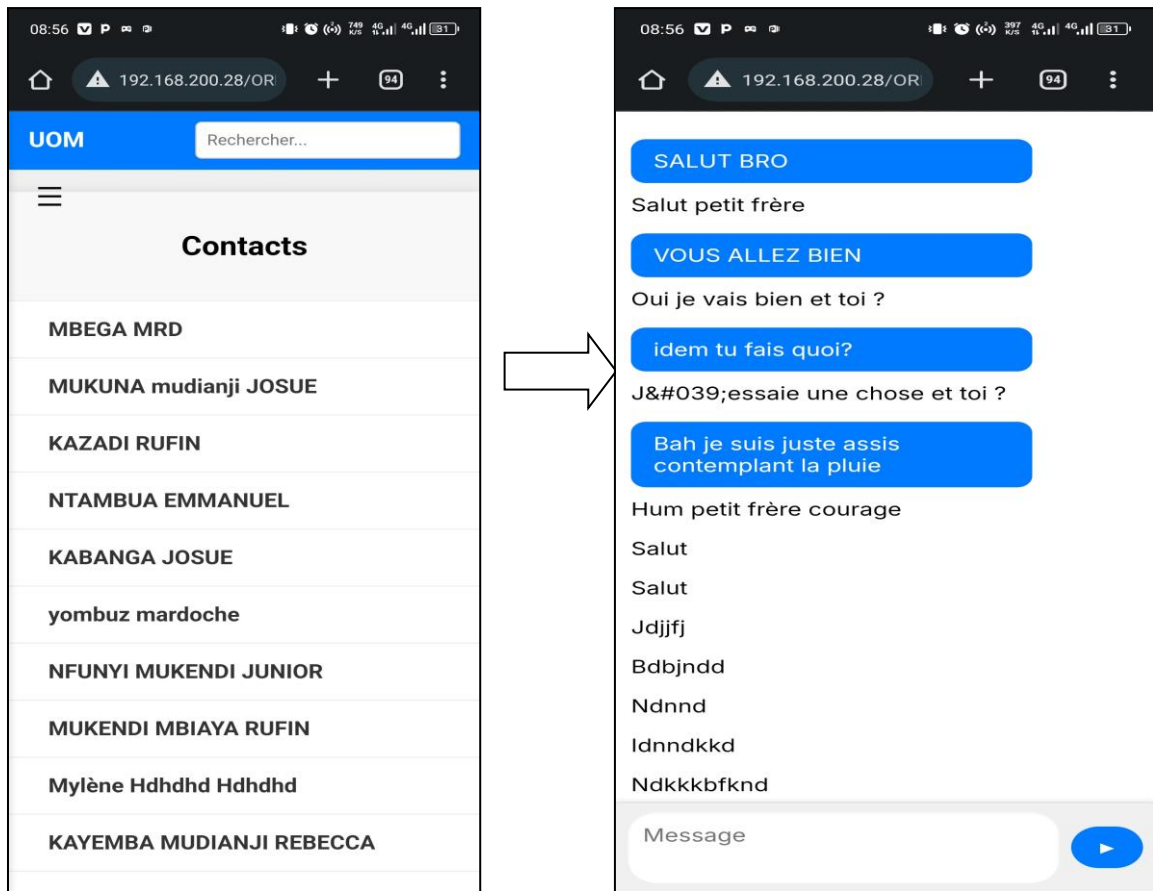


Figure 14 Interface de contact et de messagerie

5. Le Module de la délibération : Sa fonction principale est de gérer affichage des résultats de la délibération.

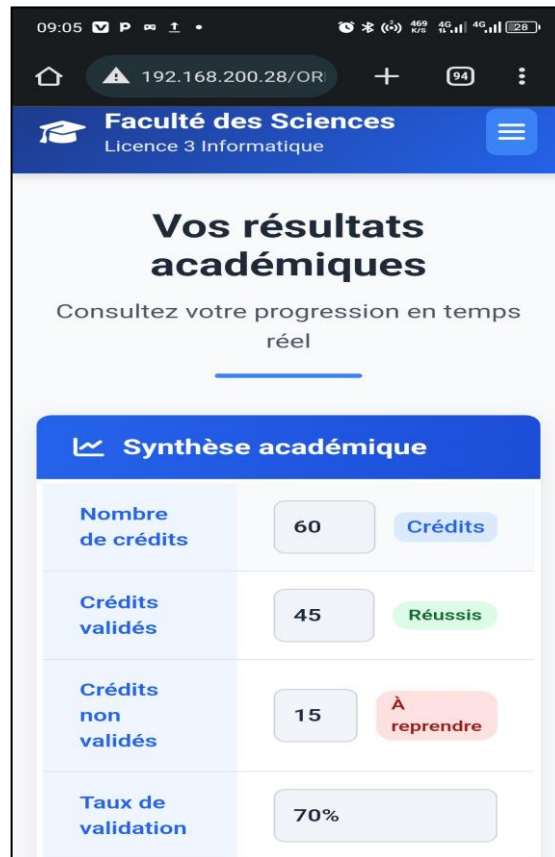


Figure 15 Interface d'affichage de résultats

6. Le Module de l'Administrateur : Sa fonction principale est de gérer publication de résultats de la délibération.



Figure 16 Interface administrateur

Pour chaque étudiant à coté e son nom il y'a une option publication, qui permet à l'administrateur de saisir les résultats pour chaque étudiant.

The screenshot displays a web browser window with the URL `localhost/ORDINATEUR/Admin1.php`. The page is titled 'Administration Délibérations - UC' and features a sidebar for 'Université Officielle de Mbujimayi' with a menu of academic levels: Licence 1 Informatique, Licence 2 Informatique, Licence 3 Informatique, Master 1 Informatique, Master 2 Informatique, Doctorat 1 Informatique, Licence 1 Droit, and Licence 1 Économie. The main content area is a modal window titled 'Délibération de MBEGA MRD' with a close button. It contains a form with the following fields and values: NOMBRE DES CREDITS (60), CREDITS VALIDE (0), CREDITS NON VALIDES (0), POURCENTAGE DE VALIDATION (0), TOTAL PONDERE (0), MOYENNE ANNUELLE (0), MENTION (0), and DECISION DU JURY (0). A green 'Enregistrer' button is at the bottom right. On the right side of the modal, there is a search bar 'Rechercher un étudiant...' and a vertical list of 'Délibérer' buttons.

Field	Value
NOMBRE DES CREDITS	60
CREDITS VALIDE	0
CREDITS NON VALIDES	0
POURCENTAGE DE VALIDATION	0
TOTAL PONDERE	0
MOYENNE ANNUELLE	0
MENTION	0
DECISION DU JURY	0

Figure 17 Interface de saisie de résultats

CONCLUSION PARTIELLE

Ce chapitre est le dernier de ce travail, il a touché au déploiement où nous avons aussi aborder la notion de l'implémentation qui consiste à transformer les modèles en une application opérationnelle matérialisant les concepts par un développement itératif, tout en incluant un flux des données entre les modules qui compose cette application.

Ainsi nous avons présenté dans ce chapitre quelques images de captures de l'implémentation de la base de données et aussi quelques images de captures de l'exécution de l'application.

CONCLUSION GENERALE

Nous voici arriver à la fin de ce travail scientifique qui est la récolte de notre semence dans l'apprentissage et la recherche scientifique qui nous a permis d'aboutir à la réalisation d'une application web mobile intégrant la gestion de la délibération, la messagerie et la file d'actualité.

Durant notre parcours dans l'élaboration de ce travail, nous avons commencé en exposant la théorie sur le système d'information, l'architecture client-serveur et enfin le web service.

Après nos études faites, nous sommes passés à la conception de l'application ; c'est grâce aux différentes technologies et méthode comme MERISE que nous sommes parvenus à la réalisation de ce travail ; Les techniques comme l'interview et documentaire sont celles qui nous ont permis à la récolte des données.

Pour tout dire, l'informatique reste aujourd'hui une science brillante dans l'épanouissement mondial et pour cela, nous rendons hommage à tous ceux qui nous ont fait de leur volonté d'accepter de nous mettre dans cette jeune science. En fin, toute œuvre humaine est imparfaite, motif pour lequel nous demanderons aux futurs chercheurs d'y apporter leur contribution

REFERENCES BIBLIOGRAPHIQUES

1. Manuel Castells « The Rise of the Network Society » (Castells, 2000).
2. Clay Shirky « Here Comes Everybody » (Shirky, 2008).
3. Creswell, J. W. (2014). Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (4th ed.). Sage Publications.
4. Bertalanffy, L. von. (1968). General System Theory: Foundations, Development, Applications. George Braziller.
5. Davenport, T. H. (1997). Information Ecology: Mastering the Information and Knowledge Environment. Oxford University Press.
6. O'Brien, James A., & Marakas, George M. (Éditions variées). Management Information Systems. McGraw-Hill/Irwin.
7. Alter, Steven. (2002). Information Systems: Foundation of E-Business. Prentice Hall.
8. O'Brien, James A., & Marakas, George M. (Éditions variées). Management Information Systems. McGraw-Hill/Irwin.
9. Tanenbaum, Andrew S., & Wetherall, David. (Éditions variées, par exemple la 6e édition). Computer Networks. Pearson.
10. L. Bass, P. Clements, et R. Kazman, Software Architecture in Practice, 3e éd. Boston, MA : Addison-Wesley Professional, 2012.
11. G. Coulouris, J. Dollimore, T. Kindberg, et G. Blair, Distributed Systems: Concepts and Design, 5e éd. Harlow, Royaume-Uni : Pearson, 2012, Chap. 2.
12. Digital Ecosystem Framework, IEEE (2022)
(<https://ieeexplore.ieee.org/document/9876543>)
13. Erl, T. (2005). Service-Oriented Architecture: Concepts, Technology, and Design. Prentice Hall PTR.

14. Booth, D., & Champlin, C. (2003). Web Services Description Language (WSDL) Version 2.0 Part 0: Primer. W3C .
15. W3C. (2003). SOAP Version 1.2 Part 1: <https://www.w3.org/TR/soap12-part1/>
16. Papazoglou, M. P. (2007). Web Services and SOA: Principles and Technology. Pearson Education.
17. Kreger, H. (2003). Web Services Conceptual Architecture (WSCA 1.0). IBM.
45. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine
46. Richardson, L., & Amundsen, M. (2013). RESTful Web APIs. O'Reilly Media .
18. Dennis, A. (2015). Systems Analysis and Design. Wiley.
19. Teorey, T. J., Yang, D., & Fry, J. P. (2011). Database Design: Know It All. Morgan Kaufmann.